

Concepts, Codes and Canonical Forms

A tutorial on ONTODAG and Formal Concept Analysis:
what they share, where they part, and why the difference
matters for AI agents and cryptographic systems

Péter Földiák

`peter.foldiak@gmail.com`

`https://github.com/petfold/ontodag`

August 2026

Abstract

ONTODAG began with an observation about brains: if a concept is represented by a set of active units in a sparse neural code, then the *overlaps* between codewords already encode a hierarchy of categories — structure that need not be carried by any synapse. That observation has a mathematical home, and it is older and richer than the observation itself: Formal Concept Analysis (FCA), a branch of applied order theory that turns exactly such a binary code — what it calls a *formal context* — into a lattice of concepts, and back again. The correspondence is a genuine bridge between a “brain” object (a code) and a “mind” object (a knowledge graph).

This tutorial is written for a reader who has met neither field. Part I builds the necessary order theory from scratch, tells the neural-code story, and then develops FCA and ONTODAG side by side on one small worked example. Part II states the correspondence precisely — ancestor-set codes *are* FCA intents, descendant cones *are* extents, and the concept lattice of an ONTODAG is its Dedekind–MacNeille completion — and then isolates the four differences that actually matter: ONTODAG has no object/attribute distinction, it stores only what was asserted rather than every closed set, its canonical form is cheap where FCA’s is exponential, and its concepts have stable names where FCA’s have none. Part III turns the comparison into two lists: seven things ONTODAG should take from FCA’s theory (implication bases as a local lint, conceptual scaling, pattern structures, stability, attribute exploration with a language model as the expert, iceberg partiality, lattice-drawing), and six it must refuse to give up. Part IV argues that these refusals are exactly what makes ONTODAG useful to AI agents and to cryptographic systems: agents need knowledge that is *canonical*, *addressable*, *verifiable* and *attributable* — four properties a language model’s knowledge conspicuously lacks — and content-addressed networks need a semantic layer whose fingerprints depend on meaning rather than on phrasing. Part V maps the research directions this opens.

Every number quoted is produced by a runnable script included with the paper; every figure is generated from its data.

A note on how to read this. Sections 2–5 are a course: they assume only school mathematics and define everything they use. A reader who already knows lattices and FCA can start at Section 6 and use Section 6.6 as a phrasebook. A reader who only wants the argument about agents and cryptography can read Section 7.7, then jump to Parts IV and V.

Contents

Part I — Foundations	5
1 Introduction	5
1.1 Two objects that turn out to be one	5
1.2 What this document is for	5
1.3 One example, used throughout	6
1.4 The thread that connects the parts	6
1.5 Reproducibility	7
2 The mathematics you need (and no more)	7
2.1 Sets, and the one relation that matters	7
2.2 Partial orders	8
2.3 Drawing a poset: Hasse diagrams	8
2.4 Directed graphs, reachability, cycles	9
2.5 Transitive closure and transitive reduction	9
2.6 Lattices: when meets and joins always exist	10
2.7 Closure operators, and the Galois connection	11
2.8 Summary of Section 2	12
3 Where the idea came from: codes in the brain	12
3.1 What a neural code is	12
3.2 The observation	13
3.3 The code matrix is a formal context	13
3.4 Why this is more than a curiosity	14
3.5 A note on Leibniz	14
3.6 Why not just use vectors?	15
4 Formal Concept Analysis from scratch	15
4.1 The formal context	15
4.2 The derivation operators	16
4.3 Formal concepts	16
4.4 The concept lattice	17
4.5 Reading a line diagram	17
4.6 The full list, and what is in it	18
4.7 Implications	19
4.8 Computing: NextClosure and complexity	19
4.9 The explosion, measured	20
4.10 The parts of FCA beyond the basics	20
4.11 What to take away	22
5 OntoDAG from scratch	22
5.1 The shape of the thing	22
5.2 Building it: <code>put</code>	23
5.3 Querying: <code>get</code>	23
5.4 The golden rule, and canonical form	25
5.5 Editing: remove, move, and cone removal	25
5.6 Merging: the CRDT property	25
5.7 Typed values: the order that is computed	26
5.8 Persistence: from a graph to 32 bytes	27
5.9 Residency: eager, lazy, sparse	27

5.10	The guarantees	27
5.11	Summary of Section 5	28
Part II — The correspondence		28
6	The dictionary: how the two formalisms correspond	28
6.1	From an ONTODAG to a formal context	28
6.2	Formal concepts are self-describing queries	29
6.3	The theorem	30
6.4	From a formal context to an ONTODAG	30
6.5	Codes and graphs are the same information	31
6.6	The complete dictionary	32
6.7	Where the analogy leaks: the meet-substitution trap	32
7	The differences that matter	33
7.1	Difference 1: one sort, not two	33
7.2	Difference 2: asserted, not closed	34
7.3	Difference 3: two kinds of canonical form	35
7.4	Difference 4: concepts with names, and concepts without	35
7.5	Difference 5: a fixed table, or a living store	36
7.6	Difference 6: finite crosses, or computed order	36
7.7	Summary: the two systems side by side	37
Part III — What each can learn from the other		37
8	What OntoDAG can learn from FCA	37
8.1	Implications and the stem base	38
8.2	Conceptual scaling, and the levels of measurement	39
8.3	Pattern structures: the theory ONTODAG already uses	39
8.4	Stability, support, and knowing which concepts are worth having	40
8.5	Boolean factor analysis and the MDL connection	41
8.6	Attribute exploration, with a language model as the expert	42
8.7	Drawing: line diagrams as a design discipline	43
8.8	Algorithms and complexity as guardrails	43
8.9	Triadic FCA, and provenance	44
8.10	What ONTODAG should refuse	44
8.11	The shopping list, ranked	45
9	What OntoDAG must keep	45
9.1	The cheap canonical form	46
9.2	Names as identity	46
9.3	Assertion over derivation	47
9.4	One query primitive	47
9.5	Mergeability	47
9.6	Exactness	48
9.7	One sort	48
9.8	The meta-commitment: a place to put derived things	48
Part IV — Why this matters now		49

10 Why this matters for AI agents	49
10.1 The four missing properties	49
10.2 This is not an argument against retrieval	50
10.3 What the agent surface actually looks like	50
10.4 Closing the world by pinning, not by decree	51
10.5 Fail-closed answers	52
10.6 Verification: from “trust the store” to “trust nobody”	52
10.7 Writing: propose, echo, confirm	52
10.8 The review problem	53
10.9 Agents sharing memory	54
10.10Token economics, briefly	54
10.11What ONTO DAG does not solve	55
11 Why this matters for cryptographic and decentralised systems	55
11.1 The gap content addressing leaves	55
11.2 Semantic content addressing	55
11.3 Convergence without consensus	56
11.4 Publishing, following, and paying rent	57
11.5 Anchoring, and settling disputes	57
11.6 Bonded claims: the economic third leg	58
11.7 Marketplaces: matching as cone intersection	58
11.8 The zero-knowledge direction	59
11.9 Governance: canonical forms, plural	59
11.10Honest limits, again	60
Part V — Directions	60
12 Directions	60
12.1 The organising picture	60
12.2 Buildable now	61
12.3 Medium term: needs a design, not a discovery	61
12.4 Longer term: genuinely open	63
12.5 The map	64
13 Conclusion	64
A Glossary	70
B Notation	73
C The experiments	74
D Colophon: how this document is built	79

Part I — Foundations

1 Introduction

1.1 Two objects that turn out to be one

Start with a picture of a brain, or at least a cartoon of one. A population of neurons responds to the things you see. For any particular thing — a face, a cup, your grandmother — some of those neurons fire and the rest stay quiet. Write down a 1 for each active neuron and a 0 for each silent one, and the thing you were looking at has become a string of bits. Do this for many things and you have a table: rows are the things, columns are the neurons, and each cell says whether that neuron fired for that thing. This table is called a *neural code*, and in the biologically realistic case it is *sparse*: only a small fraction of the neurons are active for any one stimulus (Barlow, 1972; Földiák, 1990; Olshausen and Field, 1996).

Now notice something about the table that has nothing to do with biology. Take two things — say, a spaniel and a poodle — and look at the neurons that fire for both. If a large set of neurons is shared, the two things have something in common. If the set of neurons active for “spaniel” happens to *contain* every neuron active for “dog in general”, that containment is a statement about categories: a spaniel is a dog. No wire in the brain has to represent the arrow from spaniel to dog. The arrow is already there, implicit in which codewords overlap which.

That is the observation ONTO DAG was built on (Földiák, 2003, 2026b): *a knowledge graph can be latent in a code*. The overlaps between codewords are a category structure, and if you can compute with the overlaps you can compute with the categories.

The observation has a mathematical home. It is called **Formal Concept Analysis** (FCA), it was founded by Rudolf Wille in 1982 (Wille, 1982; Ganter and Wille, 1999), and it is precisely the theory of turning a binary table into a hierarchy of concepts — and the hierarchy back into the table. FCA’s name for the table is a *formal context*; its name for the hierarchy is a *concept lattice*. What FCA contributes is not a vague analogy but an exact, invertible correspondence with forty years of theory attached: algorithms, complexity results, a theory of implications between attributes, an interactive protocol for growing a knowledge base with an expert, and a well-understood way of handling numbers and intervals.

Key idea

A neural code is a binary matrix. A binary matrix is a formal context. A formal context determines a lattice of concepts. So a code determines a concept hierarchy — a *mind* object recoverable from a *brain* object, and vice versa. This is the bridge, and FCA is the mathematics of crossing it (Endres and Földiák, 2008).

ONTO DAG is a working system that lives near this bridge but does not stand on it. It is a running piece of software — a category manager with a command-line tool, a web interface, a Python API, and a persistence layer that publishes to a decentralised storage network — and its design is shaped by engineering constraints that pure FCA never had to face: it must accept edits one at a time, it must merge the work of writers who never talk to each other, and its stored form must hash to the same 32 bytes whenever two people know the same thing. Those constraints pushed it away from FCA in specific, principled ways. This tutorial is about the resulting family resemblance: what is shared, what differs, what each side should learn from the other.

1.2 What this document is for

Four questions, in order.

- Q1. How exactly do OntoDAG and FCA correspond?** Not “both are about hierarchies” — an exact dictionary, with a theorem (Section 6).
- Q2. What can OntoDAG take from FCA’s theory?** FCA is rich and mature. ONTODAG is small and young. There is a shopping list, and some of it is affordable today (Section 8).
- Q3. What must OntoDAG *not* adopt?** Some of FCA’s central commitments would destroy the properties that make ONTODAG useful. Knowing which, and why, is more valuable than the shopping list (Section 9).
- Q4. Who benefits?** Two constituencies, in detail: AI agents (Section 10) and cryptographic / decentralised systems (Section 11). The claim is that they need the *same* four properties, and that those properties are exactly what ONTODAG’s refusals buy.

The document is written as a tutorial. It assumes no order theory, no lattice theory, no FCA, and no cryptography beyond “a hash function turns data into a short fingerprint”. Everything else is built up from school mathematics: sets, subsets, and arrows between things. Where a technical term is unavoidable it is defined at first use, boxed, and repeated in the glossary (Appendix A).

1.3 One example, used throughout

To keep the two formalisms comparable, one small example runs through the whole paper. You are planning a trip to Japan and you have six documents.

Document	What it is
outbound.pdf	the flight out; booked; refundable
inbound.pdf	the flight back; booked; not refundable
kyoto-inn.pdf	a hotel in Kyoto; booked; refundable
rail-pass.pdf	a Japan Rail pass; booked; refundable
paris-hop.pdf	an unrelated flight to Paris; booked
visa-note.md	notes on the visa; not a booking at all

There are five properties in play: Flight, Hotel, Japan, Booked, Refundable. This is deliberately tiny — small enough that every computation in Parts I and II can be checked by hand, and small enough that the concept lattice fits on a page. It is also deliberately awkward in the right way: `outbound.pdf` is a flight *and* a Japan document *and* refundable, which is exactly the situation that a folder tree cannot represent and that motivates both formalisms.

1.4 The thread that connects the parts

There is a single idea underneath all four questions, and it is worth stating before the machinery arrives, because everything later is a variation on it.

Consider what it means for a system to *know* that a spaniel is a dog. There are two ways to hold that knowledge:

- **Say it.** Store an arrow from `spaniel` to `dog`. The knowledge is an *assertion* — somebody put it there, and it is there because they did.
- **Derive it.** Store the facts about individual spaniels and individual dogs, and let the containment of one set of properties in another *produce* the arrow. The knowledge is a *consequence* — nobody put it there; it follows.

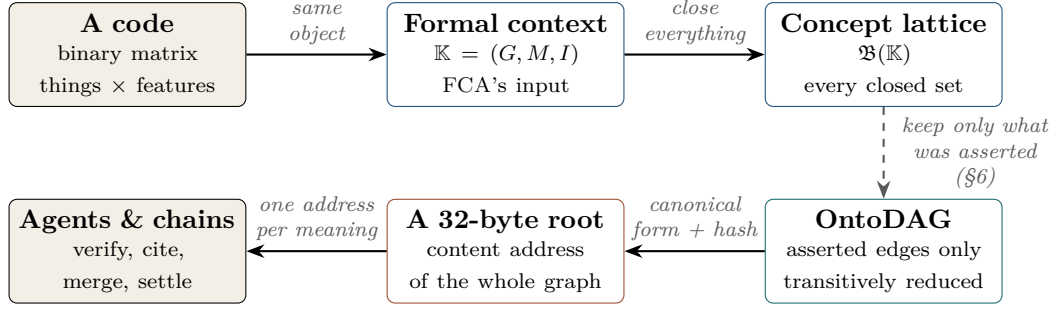


Figure 1: The road this paper travels. A neural code and a formal context are the same mathematical object seen by two communities. Closing the context gives the concept lattice, the object FCA studies. ONTODAG sits below it: the asserted fragment, kept in a unique minimal form, which is what allows the whole graph to have a single content address — and that address is what AI agents and cryptographic systems actually want.

FCA is the theory of the second option taken to its limit: given the data, compute *every* concept the data supports, and every relationship between them. ONTODAG is a system built firmly on the first: store what was asserted, in the smallest form that implies the same things, and compute the rest at query time. Nearly every difference in this paper — in cost, in canonical form, in mergeability, in whether concepts have stable names — flows from that one fork.

Neither choice is wrong. They answer different questions. FCA asks “what concepts does this data *contain*?” and gives a complete, closed, finished answer about a fixed table. ONTODAG asks “what have people said, and what follows from it right now?” and must give an answer that survives being edited, merged with a stranger’s answer, and fingerprinted for a blockchain.

Figure 1 shows the route. We will walk it from left to right.

1.5 Reproducibility

Every quantitative claim in this paper — concept counts, lattice sizes, canonical root hashes, the round-trip between codes and graphs — is computed by `paper/experiments/fca_experiments.py`, which uses only the Python standard library plus ONTODAG itself. Its full output is reproduced in Appendix C, and the figures drawn from data are generated from the same run. Where a number appears in the text it is marked with a box like this one:

Measured, not asserted

Running example: 6 documents × 5 properties, 57% of cells filled. FCA finds **10** formal concepts (out of $2^5 = 32$ conceivable property sets). The same facts as an ONTODAG: **11** nodes and **17** asserted edges.

2 The mathematics you need (and no more)

This section builds, from scratch, the handful of ideas both formalisms rest on. If you know what a lattice and a transitive reduction are, skim to Section 2.5 for the uniqueness theorem, which is the load-bearing fact of the whole paper, and then move on.

2.1 Sets, and the one relation that matters

A *set* is a collection of distinct things, written with braces: $\{a, b, c\}$. Two operations will appear on every page:

- **Intersection** $A \cap B$: the things in *both* A and B .

- **Union** $A \cup B$: the things in *either*.

And one relation, which is the real subject of this paper:

- **Containment** $A \subseteq B$: every element of A is also an element of B . We say A is a *subset* of B . If additionally $A \neq B$ we write $A \subsetneq B$ and say *strict* subset.

Containment is the prototype of everything that follows. When we say “a spaniel is a dog”, “`outbound.pdf` is a Japan document”, or “3 kg is within 0–5 kg”, we are making a statement with the same shape: one thing is a special case of another. In every formalism in this paper that statement will turn out to be, literally, a containment of sets — of properties, of members, or of numerical values.

2.2 Partial orders

Definition 2.1 (Partial order). A *partial order* on a set P is a relation $\leq$ satisfying, for all $x, y, z \in P$:

- (i) **reflexivity**: $x \leq x$;
- (ii) **antisymmetry**: if $x \leq y$ and $y \leq x$ then $x = y$;
- (iii) **transitivity**: if $x \leq y$ and $y \leq z$ then $x \leq z$.

The pair $(P, \leq)$ is a *partially ordered set*, or *poset*.

The word “partial” is the important one. In a *total* order — like the numbers on a ruler — any two elements are comparable: given x and y , either $x \leq y$ or $y \leq x$. In a partial order they need not be. **Flight** and **Hotel** are simply incomparable: neither is a special case of the other. This is not a defect of the model; it is the whole point. Reality is full of incomparable things, and any structure that forces a comparison (a filing cabinet, a folder tree, an alphabetical list) is lying about something.

Example 2.2. Containment on sets is a partial order. $\{a\} \subseteq \{a, b\}$, and $\{a\}$ and $\{b\}$ are incomparable. Divisibility on the positive integers is another: $3 \leq 12$ because 3 divides 12; 3 and 5 are incomparable. Hold on to the divisibility example — Leibniz built an entire scheme of conceptual notation on it (Section 3.5).

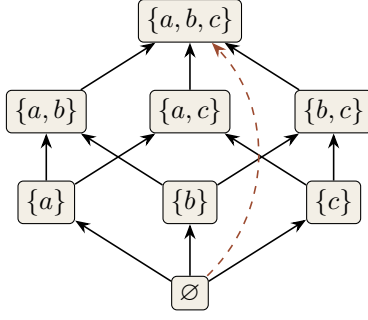
2.3 Drawing a poset: Hasse diagrams

Drawing every arrow of a partial order is hopeless: transitivity means a chain of length n needs $\binom{n}{2}$ arrows, almost all of them redundant. The standard solution is to draw only the arrows you cannot infer.

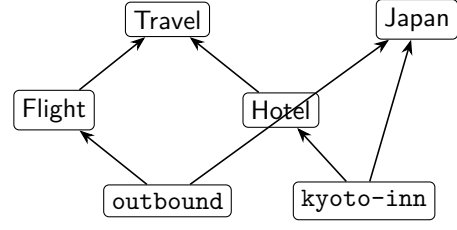
Definition 2.3 (Cover). In a poset, y *covers* x (written $x \lessdot y$) when $x < y$ and there is no z with $x < z < y$: nothing sits strictly between them.

Definition 2.4 (Hasse diagram). The *Hasse diagram* of a finite poset draws one node per element and one line per covering pair, with the greater element placed higher. All other relationships are read off by following lines upward.

A Hasse diagram is an act of compression: it stores the minimum and recovers the rest by walking. ONTO DAG does the same thing with its actual storage, and Section 2.5 is about why that is safe.



(a) All subsets of $\{a, b, c\}$ ordered by $\subseteq$. Solid arrows are covers; the dashed one is an example of the many relationships a Hasse diagram leaves implicit ($\emptyset \subseteq \{a, b, c\}$, readable by walking up).



(b) Part of the running example. `outbound` has two parents; it is a flight *and* a Japan document. No tree can draw this.

Figure 2: Hasse diagrams. Higher means more general.

2.4 Directed graphs, reachability, cycles

A *directed graph* (digraph) is a set of nodes plus a set of arrows (*edges*) between them. Unlike a poset, a digraph carries no promise of transitivity or antisymmetry — it is just arrows.

Definition 2.5 (Reachability). Node y is *reachable* from x if there is a path $x \rightarrow \dots \rightarrow y$ following arrows forwards. Every node is reachable from itself by the empty path.

Definition 2.6 (DAG). A *directed acyclic graph* (DAG) is a digraph with no cycles: you can never leave a node and return to it.

The connection to posets is the reason DAGs matter here:

Proposition 2.7. *Reachability in a DAG is a partial order. Conversely, every finite poset is the reachability order of a DAG — for instance, of its Hasse diagram.*

Reflexivity holds by the empty path, transitivity by concatenating paths, and antisymmetry is exactly acyclicity: if y were reachable from x and x from y with $x \neq y$, the concatenation would be a cycle. So a **DAG is a poset you can store**, and the acyclicity check that ONTODAG performs on every write (§5.2) is the mechanism that keeps the stored object a legitimate partial order.

Definition 2.8 (Cone). For a node x in a DAG, its *descendant cone* $\downarrow x$ is the set of nodes reachable from x , including x itself. Its *ancestor cone* $\uparrow x$ is the set of nodes from which x is reachable, again including x .

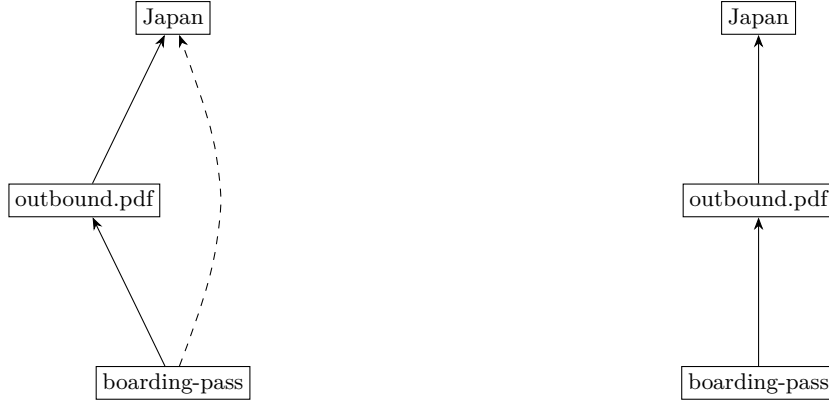
We draw more general things higher and let edges point upward, from special to general. So $\uparrow x$ — everything x is a special case of — is above x , and $\downarrow x$ — everything that is a special case of x — is below. The cone is the single most important object in this paper. In ONTODAG it is the answer to a one-word query, in FCA it is a concept’s extent, and in a neural code it is the set of stimuli that activate a given unit.

2.5 Transitive closure and transitive reduction

Two operations turn one digraph into another.

Definition 2.9 (Transitive closure). The *transitive closure* G^+ of a digraph G has the same nodes and an edge $x \rightarrow y$ whenever y is reachable from x in G . It adds every shortcut.

Definition 2.10 (Transitive reduction). A *transitive reduction* of G is a digraph G^{red} with the same nodes and the same reachability, having as few edges as possible. It removes every shortcut.



(a) As typed: someone asserted the dashed edge as well.

(b) As stored: the shortcut is dropped. Reachability is unchanged.

Figure 3: Transitive reduction in action. `boarding-pass` is still under `Japan` — via `outbound.pdf` — so storing the direct edge would add nothing but would make the stored form depend on what the user happened to type. (Figures generated with Graphviz and converted by `dot2tex`; see Appendix D.)

Closure and reduction are opposite ends of a family of graphs that all say the same thing. And now the fact that everything in ONTODAG hangs from:

Theorem 2.11 (Uniqueness of the reduction; Aho et al., 1972). *For a finite DAG, the transitive reduction exists, is unique, and is a subgraph of the original. It consists of exactly the covering pairs of the reachability order — i.e. it is the Hasse diagram.*

Why it is true, informally. Call an edge $x \rightarrow y$ *redundant* if y is reachable from x by some *other* path. In a DAG a redundant edge can always be deleted without changing reachability, because the alternative path survives the deletion — that path cannot pass through the edge $x \rightarrow y$ itself, since doing so would require returning to x , which acyclicity forbids. So deleting redundant edges one at a time terminates, and it terminates at the set of non-redundant edges, which is determined by the reachability relation alone. Determined means unique: the order of deletion cannot matter.

The caveat is worth stating, because it explains why the theorem is about *acyclic* graphs. In a graph with a cycle $x \rightarrow y \rightarrow x$, both edges are redundant given the other, and you may delete either but not both — so there are two equally minimal answers and no canonical one. $\square$

Key idea

Uniqueness is the seed of everything. Because the reduction is unique, a category structure has exactly *one* minimal representation. Two people who know the same things, having built their graphs in different orders and having typed some redundant links along the way, end up with the byte-identical graph. That is what lets ONTODAG give the whole structure a single fingerprint (§5.4), and it is why the same knowledge has the same address on a content-addressed network (§11).

2.6 Lattices: when meets and joins always exist

Posets can be missing things. In Figure 2(b), what is the most specific thing that is both a `Flight` and a `Japan` document? Looking at the picture, `outbound` is *a* thing below both — but is it *the* most specific such thing? If we later add another flight to Japan, there will be two incomparable things below both, and no single best answer. The poset has a gap where an answer ought to be.

Definition 2.12 (Meet and join). Let x, y be elements of a poset. A *lower bound* of $\{x, y\}$ is any z with $z \leq x$ and $z \leq y$. If among all lower bounds there is a greatest one, it is called the *meet*, written $x \wedge y$. Dually, the least upper bound, if it exists, is the *join* $x \vee y$.

Definition 2.13 (Lattice, complete lattice). A poset is a *lattice* if every pair of elements has both a meet and a join. It is a *complete lattice* if *every* subset — including the empty set and infinite subsets — has a meet and a join. A complete lattice automatically has a greatest element $\top$ and a least element $\perp$.

Example 2.14. The subsets of $\{a, b, c\}$ (Figure 2(a)) form a complete lattice: meet is $\cap$, join is $\cup$, $\top = \{a, b, c\}$, $\perp = \emptyset$. This is the model to keep in mind: **a lattice is a system of sets closed under intersection**, dressed up.

The lattice property is a strong demand. It says that for *any* question of the form “what is the most specific concept covering both of these?” there is a unique answer already in the structure. FCA’s central theorem guarantees exactly that (§4.4) — and paying for that guarantee, in storage and in computation, is the main cost ONTO DAG declines to pay (§7.2).

2.7 Closure operators, and the Galois connection

One more piece of machinery, because it is how FCA actually works.

Definition 2.15 (Closure operator). A map c from subsets of a set S to subsets of S is a *closure operator* if for all $A, B \subseteq S$:

- (i) **extensive**: $A \subseteq c(A)$ — closing only adds;
- (ii) **monotone**: $A \subseteq B \implies c(A) \subseteq c(B)$ — more input, more output;
- (iii) **idempotent**: $c(c(A)) = c(A)$ — closing twice is closing once.

A set with $c(A) = A$ is called *closed*.

Example 2.16. On a DAG, “take the ancestor cone of everything in A ” is a closure operator: it only adds nodes, it respects containment, and doing it twice finds nothing new because ancestry is transitive. We will meet this operator again in Section 6, where it turns out to be the ONTO DAG analogue of FCA’s closure.

Proposition 2.17. *The closed sets of a closure operator on S , ordered by $\subseteq$, form a complete lattice. The meet of a family of closed sets is their intersection; the join is the closure of their union.*

This is why lattices keep appearing: any time you have a notion of “close this set up under some rule”, the closed sets are automatically a complete lattice. FCA is, at bottom, the study of one particular closure operator.

Definition 2.18 (Galois connection). Given two sets X and Y , a pair of maps $f : \mathcal{P}(X) \rightarrow \mathcal{P}(Y)$ and $g : \mathcal{P}(Y) \rightarrow \mathcal{P}(X)$ forms a (*antitone*) *Galois connection* when

$$A \subseteq g(B) \iff B \subseteq f(A) \quad \text{for all } A \subseteq X, B \subseteq Y.$$

Both maps reverse containment ($A \subseteq A' \implies f(A') \subseteq f(A)$), and both composites $g \circ f$ and $f \circ g$ are closure operators.

The containment reversal is intuitive once seen in an example: the more properties you demand, the fewer things satisfy them; the more things you must cover, the fewer properties they all share. That trade-off *is* the Galois connection, and Section 4.2 makes it concrete.

Exercise 2.19. Check that “the set of neurons active for every stimulus in A ” and “the set of stimuli that activate every neuron in B ” form a Galois connection between stimuli and neurons. You have just derived FCA from the neural code picture — which is the content of Section 3.3.

Remark 2.20 (Further reading). The standard undergraduate text on this material is Davey and Priestley (2002); Birkhoff (1940) is the classical source. Everything above is in their first two chapters.

2.8 Summary of Section 2

Idea	Why it appears later
Containment $\subseteq$	the single shape of every “is-a” claim
Partial order	incomparable things are allowed; trees forbid them
DAG	a poset you can store; acyclicity = antisymmetry
Cone $\downarrow x$	ONTODAG’s query answer; FCA’s extent; a code’s stimulus set
Transitive reduction	unique (Thm 2.11) $\Rightarrow$ canonical form $\Rightarrow$ one hash
Lattice	every pair has a meet: FCA guarantees it, ONTODAG declines to
Closure operator	closed sets always form a lattice (Prop. 2.17)
Galois connection	the exact machine that turns a binary table into concepts

3 Where the idea came from: codes in the brain

This section is the origin story. It is not a prerequisite for the technical content — a reader in a hurry can skip to Section 4 — but it explains *why* anyone would build a system like ONTODAG in the first place, and it contains the observation that the rest of the paper formalises.

3.1 What a neural code is

A sensory neuron does not report a number to a central reader. It fires, or it does not, and what it means is determined entirely by which things in the world make it fire. Record from many neurons at once while showing many stimuli, threshold each response into “active” or “silent”, and you have a matrix:

	n_1	n_2	n_3	n_4	n_5	n_6	
face A	1	1	0	1	0	0	<i>rows: stimuli</i> <i>columns: neurons</i> <i>cell: did it fire?</i>
face B	1	1	0	0	1	0	
cup	0	0	1	0	0	1	
hand	0	1	1	0	0	1	

Real cortical codes of this kind are *sparse*: for any one stimulus only a small fraction of the units are active (Barlow, 1972; Földiák, 1990; Olshausen and Field, 1996). Sparseness is not an accident of measurement. It is what you get if you ask a network to represent its input economically — to spend few active units per item while keeping items distinguishable — and it is the representational regime in which the observation below has teeth.

Between two extremes:

- A **localist** code gives each thing its own unit — the “grandmother cell”. Simple to read, but you need as many units as things, and nothing is shared between related items.
- A **dense distributed** code has roughly half the units active for everything. Maximally efficient in bits, but the representation of one item overlaps every other for no meaningful reason.

- A **sparse** code sits between: each item activates a small set, and — crucially — *related items activate overlapping sets*.

That last clause is where structure appears.

3.2 The observation

Take the code for “spaniel” and the code for “dog”. In a sparse code that has learned anything about the world, the units active for a spaniel will include the units that respond to dogs in general, plus some extra ones specific to spaniels. Write $\text{code}(x)$ for the set of active units of stimulus x . Then

$$\text{code}(\text{dog}) \subseteq \text{code}(\text{spaniel}).$$

Read that line slowly, because it is the whole idea. The taxonomic statement “a spaniel is a dog” has become a containment between two sets of active units. **More specific means more bits on.** And containment brings its entire order theory (Section 2) along with it: the codes are partially ordered by $\subseteq$, incomparable things are exactly things neither of which is a special case of the other, and the covering relation of that order is a Hasse diagram — a category graph.

Key idea

The category graph need not be stored anywhere. No synapse has to encode the arrow from spaniel to dog. The arrow is a *consequence* of which units overlap. Associations between concepts can be implicit in the code rather than carried by connections (Földiák, 2003).

This has three immediate consequences that shaped ONTODAG’s design, and each of them will reappear later in the paper wearing engineering clothes.

1. **Queries are set operations.** “Find everything that is both a dog and brown” becomes: find every x whose code contains the dog units *and* the brown units. A conjunctive query is an intersection — or, in hardware terms, a bitwise AND. This is ONTODAG’s only query primitive (§5.3).
2. **Similarity is overlap.** Two items with largely overlapping codes are similar without anyone having said so. Graded retrieval — “what is most like this?” — is $|\text{code}(x) \cap \text{code}(y)|$, computed on the same representation as exact retrieval, with a threshold as the only difference (Földiák, 2003).
3. **Generalisation is a subset.** A concept more general than x is a *subset* of x ’s code. Dropping bits is abstracting. This is why Borges’s Funes, who remembered every instant in full detail and could therefore drop nothing, was “virtually incapable of general, platonic ideas” (Borges, 1952): no compression, no concepts.

3.3 The code matrix is a formal context

Here is the bridge, stated plainly. Everything in Section 4 is a development of this single sentence:

Key idea

A binary stimulus $\times$ neuron matrix and a *formal context* $\mathbb{K} = (G, M, I)$ are the same object. Objects G are the stimuli, attributes M are the neurons, and the incidence I says “this neuron fired for this stimulus”. Everything FCA proves about formal contexts is therefore a theorem about neural codes.

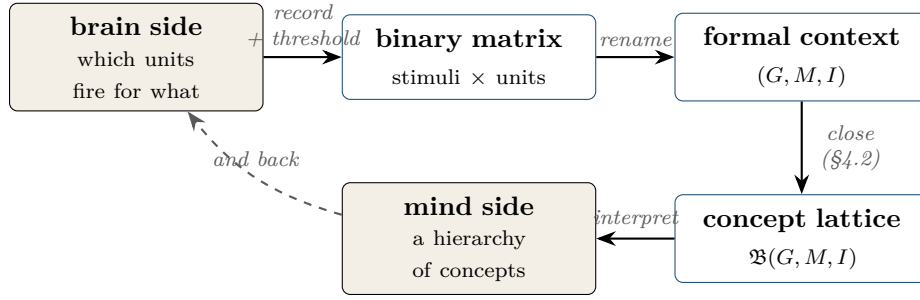


Figure 4: The mind–brain round trip that motivates the whole exercise. The left column is neuroscience, the right column is mathematics, and the two middle arrows are pure renaming. Because the maps in Section 4.2 are invertible, the trip can be made in either direction: a code determines a concept hierarchy, and a concept hierarchy can be embedded into a code.

This is not a metaphor invented for this paper; it is a published research programme. Endres and Földiák (2008) applied FCA to recordings from macaque superior temporal sulcus, building the formal context from spike counts by an exact Bayesian thresholding procedure and then reading the concept lattice as a description of what the neural population represents. The lattice exposed a hierarchical organisation of face representations and evidence for a product-of-experts style code — structure that is invisible in the raw firing rates and that no dimensionality reduction into a Euclidean space would have expressed, because the structure is *ordinal*, not metric. The follow-up (Endres et al., 2010) pushed the same machinery toward semantic decoding.

3.4 Why this is more than a curiosity

Two directions of travel, both useful.

Code → concepts (interpretation). Given a recorded population code, the concept lattice tells you what the population distinguishes and how its distinctions nest. This is a tool for neuroscience, and it has the attractive property of being *non-parametric about geometry*: it assumes nothing about distances, only about which units fire.

Concepts → code (design). Given an ontology you want a system to hold, you can embed it in a binary code such that subsumption is subset containment. This is the direction ONTO DAG cares about, and Section 6 shows it is exact: the reflexive ancestor set of each node *is* such a code, and the graph can be reconstructed from the codes with nothing lost. Historically the same idea has been rediscovered several times: as bitvector encodings of type lattices for fast subsumption checks in programming languages (Ait-Kaci et al., 1989); as faceted classification in library science, where Ranganathan (1933) proposed describing a work by a *set* of independent facets rather than one shelf path — the librarians’ independent discovery that the lattice, not the tree, is the honest shape; as sparse distributed representations and hyperdimensional computing (Kanerva, 2009; Plate, 1995); and, three centuries earlier, as Leibniz’s prime-number notation.

3.5 A note on Leibniz

Leibniz’s scheme deserves a paragraph, because it is the same lattice written in a different alphabet and it makes the algebra vivid (Leibniz, 1666). Assign a distinct prime to each primitive concept, and give a composite concept the *product* of its constituents’ primes. If *animal* = 2 and *rational* = 3, then *human* = 6. Now:

$$A \text{ subsumes } B \iff \text{the number of } A \text{ divides the number of } B.$$

Divisibility of square-free numbers is exactly set containment on the sets of prime factors. So Leibniz’s “let us calculate” and the bitwise-AND of two cone bitmaps are the same operation in different notation, three and a half centuries apart. What Leibniz lacked was not the algebra but a *source* for the primitives; FCA supplies one (derive them from data), and ONTODAG supplies another (let people assert them, and merge what they assert).

3.6 Why not just use vectors?

A reader who has met machine learning will ask why one would use sets of bits rather than real-valued embeddings, which are what modern systems actually use. The honest answer is that they solve different problems, and the difference is the theme of this paper.

	Sparse binary code / DAG	Dense real embedding
Subsumption	exact set containment, decidable	approximate, thresholded
Similarity	overlap count, exact	cosine distance, learned
Query	intersection: complete and sound	nearest neighbours: neither
Novel item	new bit, no retraining	re-embed, possibly retrain
Explanation	the shared bits <i>are</i> the reason	post-hoc at best
Identity	a name; stable	a vector; changes when the model does
Merging two	union of assertions, converges	incomparable spaces

Embeddings are superb at recall from vague input, which is precisely what an exact structure cannot do. Discrete structures are superb at exactness, verifiability and merging, which is precisely what an embedding cannot do. Section 10 argues that a practical agent memory wants both, with the embedding proposing and the DAG disposing — and that the interesting engineering is in the interface between them, not in a winner.

Try it yourself

Take the four-stimulus code at the start of this section. Which stimuli activate a superset of $\{n_2, n_3\}$? (Answer: only *hand*.) Which pairs of stimuli have overlapping codes? Sketch the containment order on the four codewords — you will find that *face A* and *face B* are incomparable, sharing $\{n_1, n_2\}$, which is the code of a concept that has no name in the table but plainly ought to. That nameless shared concept is a formal concept, and finding all of them is what Section 4 is about.

4 Formal Concept Analysis from scratch

FCA was introduced by Wille (1982) as a way of “restructuring lattice theory” — taking an abstract branch of algebra and giving it a concrete interpretation in terms of objects, attributes and concepts, so that lattice diagrams become something a domain expert can read. The standard reference is Ganter and Wille (1999); Carpineto and Romano (2004) and Priss (2006) are gentler surveys.

This section develops the theory on the running example of Section 1.3. Everything is computed rather than asserted: the numbers come from `fca_experiments.py` (Appendix C).

4.1 The formal context

Definition 4.1 (Formal context). A *formal context* is a triple $\mathbb{K} = (G, M, I)$ where G is a set of *objects*, M is a set of *attributes*, and $I \subseteq G \times M$ is an *incidence relation*. We write gIm for “object g has attribute m ”.

Table 1: The running example as a formal context $\mathbb{K} = (G, M, I)$: six documents, five properties, 57% of the cells filled.

	<i>Flight</i>	<i>Hotel</i>	<i>Japan</i>	<i>Booked</i>	<i>Refundable</i>
outbound.pdf	×		×	×	×
inbound.pdf	×		×	×	
kyoto-inn.pdf		×	×	×	×
rail-pass.pdf			×	×	×
paris-hop.pdf	×			×	
visa-note.md			×		

The letters are from the German: *Gegenstände* (objects), *Merkmale* (attributes). A context is just a cross table, and our running example is this one:

Note what a context does *not* contain. There is no hierarchy in it — no statement that *Flight* is a kind of anything, no statement that **outbound.pdf** is more specific than **inbound.pdf**. The context is raw co-occurrence. Everything hierarchical that follows will be *derived* from it. That is FCA’s central promise and, as we shall see, also its central cost.

4.2 The derivation operators

Definition 4.2 (Derivation). For a set of objects $A \subseteq G$ and a set of attributes $B \subseteq M$:

$$A' = \{m \in M : g I m \text{ for all } g \in A\}, \quad B' = \{g \in G : g I m \text{ for all } m \in B\}.$$

A' is “the attributes shared by all of A ”; B' is “the objects having all of B ”. Both are written with the same prime, and which one is meant is determined by the argument.

Example 4.3. In Table 1:

$$\begin{aligned} \{\text{outbound.pdf}, \text{inbound.pdf}\}' &= \{\text{Flight}, \text{Japan}, \text{Booked}\}, \\ \{\text{Flight}, \text{Japan}\}' &= \{\text{outbound.pdf}, \text{inbound.pdf}\}, \\ \{\text{Hotel}\}' &= \{\text{kyoto-inn.pdf}\}, \\ \emptyset' &= M \quad (\text{everything is shared by no objects at all}). \end{aligned}$$

The two maps form an (antitone) Galois connection in the sense of Section 2.7: enlarging A can only shrink A' , and $A \subseteq B' \iff B \subseteq A'$. Both composites are closure operators. We write $A'' = (A')'$ and call it the closure of A .

Watch out

The prime is not an inverse. A'' generally contains more than A : it is A plus every other object that happens to share all of A ’s common attributes. Try $A = \{\text{outbound.pdf}\}$: $A' = \{\text{Flight}, \text{Japan}, \text{Booked}, \text{Refundable}\}$ and $A'' = \{\text{outbound.pdf}\}$ again — but $A = \{\text{visa-note.md}\}$ closes to $A'' = \{\text{outbound.pdf}, \text{inbound.pdf}, \text{kyoto-inn.pdf}, \text{rail-pass.pdf}, \text{visa-note.md}\}$, because *visa-note*’s only attribute is *Japan* and all those documents have it.

4.3 Formal concepts

Definition 4.4 (Formal concept). A *formal concept* of $\mathbb{K}$ is a pair (A, B) with $A \subseteq G$, $B \subseteq M$, $A' = B$ and $B' = A$. A is the *extent* (the things falling under the concept), B the *intent* (the properties defining it).

The two conditions are a fixed-point demand, and they encode a very old intuition about what a concept is: *the extent and the intent must determine each other exactly*. A concept is not any old pair of a set of things and a set of properties. It is a pair in perfect balance: every object in the extent has every attribute in the intent (that much is easy), and crucially there is no object outside the extent that would also qualify, and no attribute outside the intent that all the extent happens to share.

Example 4.5. $(\{\text{outbound.pdf}, \text{inbound.pdf}\}, \{\text{Flight}, \text{Japan}, \text{Booked}\})$ is a formal concept: the two flights to Japan share exactly those three properties, and no other document has all three. Informally, it is the concept *Japanese flight booking*, which nobody put in the table — it emerged.

By contrast $(\{\text{outbound.pdf}\}, \{\text{Flight}\})$ is *not* a concept: `outbound.pdf` has more attributes than just `Flight`, so the pair is unbalanced.

Proposition 4.6. (A, B) is a formal concept iff A is a closed object set ($A'' = A$) and $B = A'$; equivalently iff B is a closed attribute set ($B'' = B$) and $A = B'$. So concepts are in bijection with closed attribute sets, which is how algorithms enumerate them.

4.4 The concept lattice

Order concepts by specificity: $(A_1, B_1) \leq (A_2, B_2)$ when $A_1 \subseteq A_2$ — equivalently, when $B_2 \subseteq B_1$. Fewer things, more properties, further down. The set of all concepts under this order is written $\mathfrak{B}(\mathbb{K})$.

Theorem 4.7 (Basic Theorem of FCA; Wille, 1982; Ganter and Wille, 1999). $\mathfrak{B}(\mathbb{K})$ is a complete lattice, in which

$$\bigwedge_{t \in T} (A_t, B_t) = \left(\bigcap_t A_t, \left(\bigcup_t B_t \right)'' \right), \quad \bigvee_{t \in T} (A_t, B_t) = \left(\left(\bigcup_t A_t \right)'', \bigcap_t B_t \right).$$

Conversely, every complete lattice is isomorphic to the concept lattice of some context. So “complete lattice” and “concept lattice” are the same notion up to isomorphism.

Two things are worth extracting from that statement, one reassuring and one expensive.

Reassuring: meets are intersections of extents. “The most specific concept that is both a `Flight` and a `Japan` document” always exists, is unique, and is computed by intersecting. That is a strong guarantee, and it is exactly what makes concept lattices navigable.

Expensive: *completeness* means *every* subset has a meet, including subsets nobody asked about. The lattice contains a concept for every closed attribute set, whether or not any object realises it, whether or not anybody would recognise it, and whether or not the co-occurrence that produced it is a real regularity or a coincidence. Section 4.9 measures what that costs.

4.5 Reading a line diagram

Figure 5 shows the lattice. The convention deserves explanation because it is initially surprising and then very economical.

In *full labelling* you would write each concept’s whole extent and whole intent inside its node, which repeats every name many times. *Reduced labelling* writes each name once, at the concept where it is “introduced”:

- attribute m labels the concept (m', m'') — the *largest* concept whose intent contains m ;
- object g labels the concept (g'', g') — the *smallest* concept whose extent contains g .

and then the reading rules recover everything:

- the **intent** of a concept is every attribute label reachable by walking *up*;

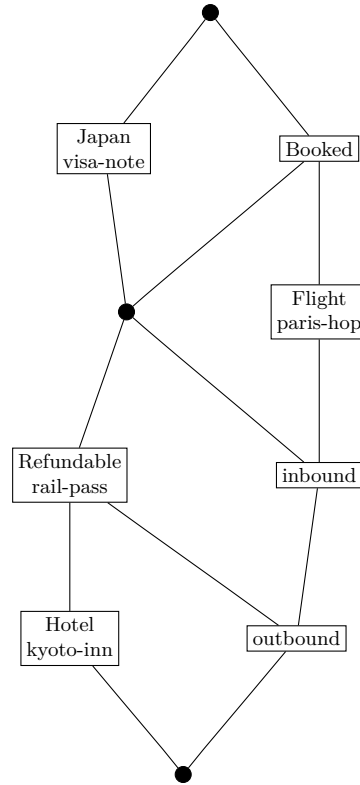


Figure 5: The concept lattice of Table 1: 10 concepts, 13 covering edges, drawn with FCA’s standard *reduced labelling*. Each attribute and each object name appears exactly once. To read off whether an object has an attribute, start at the object’s node and walk *upward* along any lines: you have the attribute iff you can reach its label. The black disc is a concept that introduces neither a new attribute nor a new object. Diagram generated with Graphviz from the computed lattice.

- the **extent** is every object label reachable by walking *down*;
- “object g has attribute m ” iff you can walk up from g ’s label to m ’s label.

Try it yourself

In Figure 5, find **rail-pass** and walk upward. You should reach **Refundable**, **Japan** and **Booked**, but not **Flight** or **Hotel**. Compare with Table 1. Then find the concept where **Booked** sits and walk *down*: you collect all five booked documents — and, satisfyingly, not **visa-note**.

4.6 The full list, and what is in it

Here is every concept of the running example, computed and sorted by extent size. **Supp** is the extent size (in data-mining language, the support); **stab** is Kuznetsov’s *stability* (Kuznetsov, 2007), the fraction of subsets of the extent that still close to the same intent — a measure of how much the concept depends on any particular object.

Concepts 0 and 9 are the lattice’s $\top$ and $\perp$: a complete lattice must have them, so they are manufactured whether or not anything in the world corresponds. Concept 9 is the concept of a document that is simultaneously a flight and a hotel — empty, and necessarily so. Concepts 3, 4, 5 and 6 are the interesting ones: *nobody wrote them down*, and each is a genuine category (“booked Japan documents”, “booked flights”, “refundable Japan bookings”, “booked flights to Japan”). This is FCA doing what it is for.

Table 2: All 10 formal concepts of Table 1. Only 10 of the $2^5 = 32$ conceivable attribute sets are closed.

#	Intent	Extent	supp	stab
0	$\emptyset$	all six documents	6	0.25
1	Booked	all but visa-note	5	0.38
2	Japan	all but paris-hop	5	0.50
3	Booked, Japan	outbound, inbound, kyoto-inn, rail-pass	4	0.38
4	Booked, Flight	outbound, inbound, paris-hop	3	0.50
5	Booked, Japan, Refundable	outbound, kyoto-inn, rail-pass	3	0.62
6	Booked, Flight, Japan	outbound, inbound	2	0.50
7	Booked, Flight, Japan, Refundable	outbound	1	0.50
8	Booked, Hotel, Japan, Refundable	kyoto-inn	1	0.50
9	all five attributes	$\emptyset$	0	1.00

4.7 Implications

Definition 4.8 (Attribute implication). For $B, C \subseteq M$, the implication $B \rightarrow C$ holds in $\mathbb{K}$ if every object having all attributes in B also has all attributes in C ; equivalently, if $C \subseteq B''$.

Measured, not asserted

The single-premise implications of Table 1:

Flight $\rightarrow$ Booked, Refundable $\rightarrow$ Booked, Japan, Hotel $\rightarrow$ Booked, Japan, Refundable.

Read them aloud and the value is obvious: “every flight here is booked”, “everything refundable is a booked Japan document”. Read them again and the danger is equally obvious: these are facts about *this table*, not laws. One unbooked flight destroys the first. Section 8.1 is about exactly this distinction, and it is the single most important thing ONTODAG must get right if it borrows anything from this part of FCA.

The set of all implications holding in a context is infinite (it is closed under trivial consequences), but it has a canonical finite generating set:

Theorem 4.9 (Duquenne–Guigues base; Guigues and Duquenne, 1986). *Every formal context has a unique minimum-cardinality set of implications from which all others follow, the stem base. Its premises are the pseudo-intents of the context.*

The stem base is a *canonical* object: same context, same base. It is the closest thing FCA has to ONTODAG’s canonical form, and comparing the two notions of canonicity (Section 7.3) is one of the more illuminating exercises in this paper.

4.8 Computing: NextClosure and complexity

The standard enumeration algorithm is Ganter’s *NextClosure* (2010). Fix an order on the attributes. Every closed set has a successor in *lectic* order (a variant of alphabetical order on the characteristic bit vectors), computable directly. Starting from $\emptyset''$ and repeatedly taking successors enumerates every closed set, in order, using memory for one set at a time — no lattice needs to be held in RAM. The paper’s implementation (Appendix C) is thirty lines and is checked against a brute-force oracle on every example in this document.

Many faster algorithms exist — Close-by-One and its descendants, Lindig’s, Titanic — and they are compared empirically by Kuznetsov and Obiedkov (2002) and Krajca et al. (2010). None of them can escape the following.

Theorem 4.10 (Hardness; Kuznetsov, 2001). *The number of formal concepts of a context can be exponential in the size of the context, and determining that number is #P-complete.*

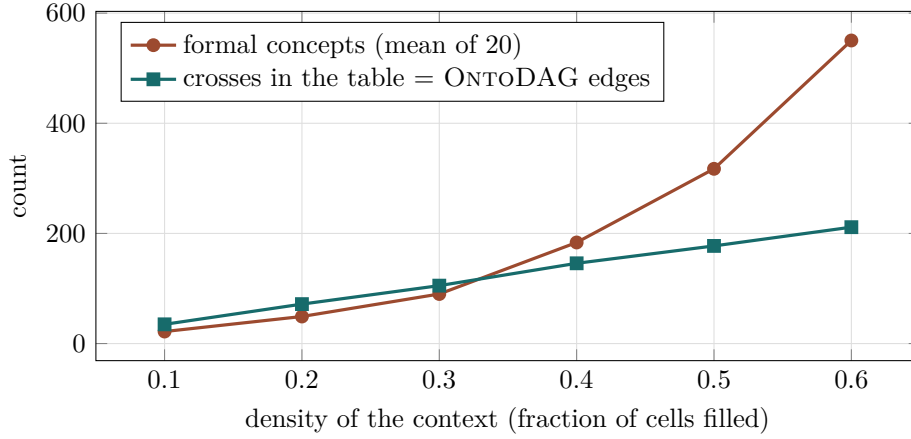


Figure 6: Random contexts, 30 objects $\times$ 12 attributes. The stored data grows linearly with density; the concept lattice does not. At density 0.6 the table has about 211 crosses and the lattice has about 550 concepts — and this is a *small* context. Data from `experiments/explosion.dat`, generated by the experiment script.

There is nothing pathological about the exponential case. It is reached by the simplest imaginable table:

Definition 4.11 (Contranominal scale). $\mathbb{N}_n^c = (\{1, \dots, n\}, \{1, \dots, n\}, \neq)$: n objects, n attributes, and object i has attribute j exactly when $i \neq j$. Every attribute subset is closed, so $|\mathfrak{B}| = 2^n$.

4.9 The explosion, measured

Measured, not asserted

Contranominal scales, computed:

n	concepts	ONTO DAG nodes	ONTO DAG edges
4	16	8	12
8	256	16	56
10	1 024	20	90
12	4 096	24	132

And on random data, where the effect is less extreme but still decisive. Figure 6 plots the mean number of concepts of a random context with 30 objects and 12 attributes, against the number of crosses in the table (which is what ONTO DAG would store as edges), as the density rises.

This is not a criticism of FCA. Closing a context is an expensive operation because *the information is really there* — those concepts are genuinely supported by the data. It is, however, decisive for a system that must hold a growing store in a browser tab, hash it after every edit, and merge it with a stranger’s copy. Section 7.2 returns to this.

4.10 The parts of FCA beyond the basics

FCA has grown a substantial toolbox in forty years. Six branches matter for this paper; each reappears in Section 8 as a candidate for adoption.

Many-valued contexts and conceptual scaling. Real data has numbers, not crosses. A *many-valued context* (G, M, W, I) assigns each object a value in W for each attribute; *conceptual scaling* converts it into an ordinary context by choosing, per attribute, a *scale* that says which

comparisons matter (Ganter and Wille, 1999, ch. 1). The standard scales are worth memorising because they are a checklist of ways to turn a value into crosses:

Scale	Turns a value into	Example
nominal	one attribute per distinct value, mutually exclusive	colour
ordinal	“ $\leq v$ ” attributes, one per threshold	school grade
interordinal	both “ $\leq v$ ” and “ $\geq v$ ”	temperature ranges
biordinal	two ordinal scales meeting in the middle	good/bad axes
contranominal	“ $\neq v$ ” — the exponential one	(avoid)

Pattern structures. Scaling is lossy and arbitrary. Pattern structures (Ganter and Kuznetsov, 2001) generalise FCA so that an object’s description need not be a set of attributes at all: it can be any element of a meet-semilattice with a similarity operation $\sqcap$. The most-used instance is the *interval pattern structure*, where a description is an interval and $\sqcap$ is the convex hull (Kaytoue et al., 2011). Remember this; ONTODAG’s handling of typed values (§5.7) turns out to be an instance of it that was arrived at independently.

Iceberg lattices and frequent closed itemsets. Keep only concepts whose extent exceeds a support threshold (Stumme et al., 2002). The result is the top of the lattice — the “iceberg” — and it is exactly the set of frequent closed itemsets from association-rule mining (Pasquier et al., 1999). A principled way to be partial.

Stability. A different filter: keep concepts that survive the removal of objects (Kuznetsov, 2007). Support asks “how many?”; stability asks “how robust?”. The two disagree usefully, as Table 2 shows — concept 5 has lower support than concept 3 but higher stability.

Boolean factor analysis. Bělohlávek and Vychodil (2010) proved that formal concepts are exactly the optimal factors for exact Boolean matrix decomposition: to write a binary matrix as a product of two binary matrices with as few inner dimensions as possible, use concepts. This is the result that connects FCA to the compression-based learning of Section 8.5: if concepts are factors, then choosing *which* concepts to keep is a model-selection problem, and MDL is the principled answer (Miettinen and Vreeken, 2014; Földiák, 2026a).

Attribute exploration. An interactive protocol (Ganter and Obiedkov, 2016): the system computes the next implication that its current data does not refute, and asks a domain expert “is this always true?”. The expert either confirms — and the implication joins the knowledge base — or supplies a counterexample, which is added as an object. Repeat. On termination the base is complete for the domain, not merely for the sample. Section 8.6 argues that this forty-year-old protocol has just acquired an ideal new participant.

Further branches, in one line each. *Triadic* FCA adds a third dimension — objects $\times$ attributes $\times$ conditions (Lehmann and Wille, 1995). *Fuzzy* FCA grades the incidence (Bělohlávek, 2004). *Relational* Concept Analysis handles links between objects of different sorts by iterated scaling of relations (Rouane-Hacene et al., 2013). Applications range from software refactoring (Snelling and Tip, 2000) to a broad survey of knowledge-processing uses (Poelmans et al., 2013).

4.11 What to take away

Key idea

FCA is **derivation to completion**. Give it a binary table and it returns every concept the table supports, organised into a complete lattice, with a canonical implication base and a mature algorithmic toolbox. It answers the question “*what concepts does this data contain?*” exhaustively and exactly.

Its costs are the direct price of that completeness: the answer can be exponentially larger than the question; it describes one fixed table, so adding a single object can restructure the whole lattice; and its concepts have no names — a concept *is* its extent and intent, so when the data changes, “the same” concept may not exist any more.

Those three costs are precisely the three problems a live, shared, editable, fingerprinted category store must not have. Section 5 shows what a system looks like when it is designed around avoiding them.

5 OntoDAG from scratch

Now the other side. ONTODAG (Földiák, 2026b) is a working system: a Python package, a command-line tool `odag`, a web interface, an agent interface, and a persistence layer that publishes to content-addressed storage. This section develops it on the same six documents, with real transcripts.

5.1 The shape of the thing

ONTODAG stores one graph. Nodes have names; edges mean “is-under”, with the more general node above. There is one distinguished node, the root `*`, which is above everything. Edges may converge — a node may have any number of parents — so the graph is a DAG rather than a tree, and by Proposition 2.7 it is a stored partial order.

The first design decision is a subtraction, and it is the one that most distinguishes ONTODAG from FCA:

Key idea

There is no distinction between objects and attributes. Everything is a node. `Flight` is a node; `outbound.pdf` is a node; `Travel` is a node. A node may have children today and none tomorrow. “Category” and “item” are roles a node plays in a particular sentence, not types in the data model.

This is a deliberate contrast with both neighbours. The relational model (Codd, 1970) fixes a schema per relation, which suits large collections of homogeneous records rather than an open-ended universe of heterogeneous things; description-logic ontology stacks (Gruber, 1993; W3C OWL Working Group, 2012) draw a firm line between classes and individuals and buy a great deal of expressiveness with it. ONTODAG declines both, and Section 7.1 weighs what that costs.

The consequence is that categories nest without any extra machinery. `Flight` can be under `Travel` in the same way that `outbound.pdf` is under `Flight`. In FCA, by contrast, an attribute is an attribute forever: it lives in M , it cannot be an element of G , and if you want a hierarchy *of attributes* you must build it outside the formalism or encode it by scaling. Section 7.1 develops this comparison properly.

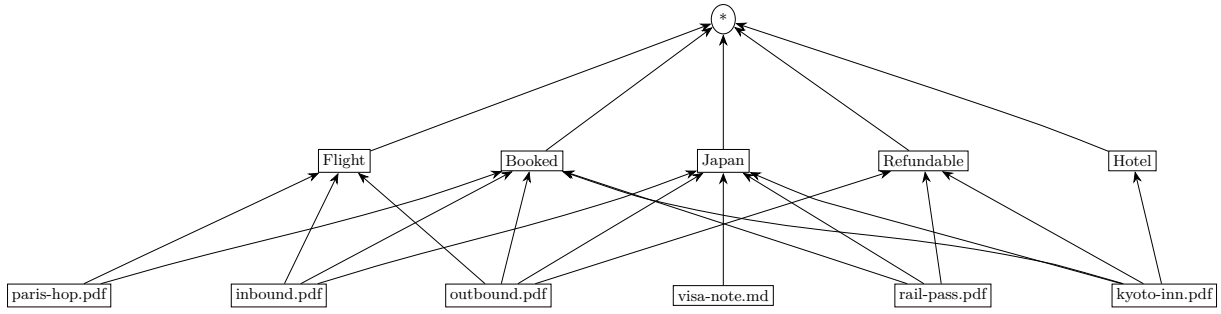


Figure 7: The running example as an ONTO DAG, drawn from the live graph. Grey nodes are the ones we would call categories, white ones the documents — but that is a distinction in the reader’s head, not in the data. Arrows point from special to general. Compare with the concept lattice of Figure 5: same facts, different object.

5.2 Building it: put

The write primitive is `put( $x, \{p_1, \dots, p_k\}$ )`: place node x under each of the given parents, creating x if needed. Here is the running example, actually typed:

```
$ odag put Travel '*'
$ odag put Japan '*'
$ odag put Booked '*'
$ odag put Refundable '*'
$ odag put Flight Travel
$ odag put Hotel Travel
$ odag put outbound.pdf Flight Japan Booked Refundable
$ odag put inbound.pdf Flight Japan Booked
$ odag put kyoto-inn.pdf Hotel Japan Booked Refundable
$ odag put rail-pass.pdf Japan Booked Refundable
$ odag put paris-hop.pdf Flight Booked
$ odag put visa-note.md Japan
```

(Silence means success: `odag` follows the Unix convention that a tool says nothing when it works.)

For each requested edge $p \rightarrow x$, `put` performs four checks in order, and the order matters:

1. **Already implied?** If x is already reachable from p by another path, the edge adds no information — skip it silently.
2. **Would it create a cycle?** If p is reachable from x , adding $p \rightarrow x$ would close a loop. Reject the *whole operation* with an error, before modifying anything. This is what keeps the stored graph a partial order (antisymmetry = acyclicity).
3. **Does it make an old edge redundant?** If some ancestor of p has a direct edge to x , that edge is now implied by the new one — delete it.
4. Add the edge.

Steps 1 and 3 together maintain the transitive reduction incrementally. Step 3 is subtler than it looks: the redundancy can appear in either direction (the new edge may bypass an existing edge from above *or* from below), and getting only one direction is a real bug that ONTO DAG shipped and later fixed — with the consequence, discussed in Section 5.6, that merge stopped being order-independent until it was corrected.

5.3 Querying: get

The entire query language is one operation.

Definition 5.1 (The query primitive).

$$\text{get}(A_1, \dots, A_k) = \downarrow A_1 \cap \dots \cap \downarrow A_k$$

— everything that is below *all* of the given categories.

```
$ odag get Japan Flight
inbound.pdf
outbound.pdf

$ odag get Japan Refundable
kyoto-inn.pdf
outbound.pdf
rail-pass.pdf

$ odag get Flight Hotel          # nothing is both
$ odag get Travel
Flight
Hotel
inbound.pdf
kyoto-inn.pdf
outbound.pdf
paris-hop.pdf
```

Three details are worth noticing in that transcript.

First, `get Travel` returns `Flight` and `Hotel` alongside the documents. Cones do not stop at some notional item level, because there is no item level. This is the one-sorted design showing through, and it is a feature: “show me everything travel-related” should include the sub-categories.

Second, `get Flight Hotel` returns nothing, and that is an answer, not an error. An empty intersection is a perfectly good statement about the world.

Third, the empty query `get` with no arguments returns *everything*: the intersection of no cones is unconstrained, which makes the operation total and makes `get`, `list` and `get '*'` one code path.

Two derived operations complete the query surface. `get_any` takes several such conjunctions and unions the results (disjunctive normal form, computed entirely at query time, never stored). And `is_below(x, y)` answers the yes/no question directly, by walking *upward* from *x* — ancestor chains are short even in huge graphs, so this never touches a cone:

```
$ odag below outbound.pdf Travel
true
$ echo $?
0
```

The planner. A naive `get` would compute every cone and intersect. ONTODAG does better, in ways borrowed from database query optimisers and all *provably result-preserving*: it drops query terms that are ancestors of other terms (they constrain nothing extra), orders the remaining cones smallest-first using the exact `descendant_count` stored on every node, and then — knowing at runtime how many candidates survive each step — chooses between *walking* the next cone and *probing* upward from each surviving candidate. It stops the moment the running result is empty. The maxim is: *plan with what is known in advance, adapt on what can only be learned by doing, and never let either change the answer.*

```
$ odag count Japan
5
```

That `descendant_count` is an exact, always-maintained `COUNT(*)` per category, and it is maintained by *delta* rather than recomputation — appending a fresh node adds exactly one to each distinct ancestor, contraction removes exactly one, and dropping a redundant edge changes nothing by definition. It is also stored inside each record, hence hashed, hence it must be an exact function of content and not of history.

5.4 The golden rule, and canonical form

Key idea

Keep no link you can infer. ONTODAG stores, at all times, the transitive reduction of the asserted relation. By Theorem 2.11 that graph is *unique*, so the stored form depends only on *what is known*, never on the order it was typed in or on how many redundant links the user tried to add.

This is easy to state and easy to underrate. It means the graph has a *canonical form*, and a canonical form can be fingerprinted.

Measured, not asserted

Fifty random build orders of the same six facts, each replayed into a fresh ONTODAG: **1** distinct graph. Ten of those orders replayed through the content-addressed persistence layer: **1** distinct root hash,

5ff08fead6474d5e9cb1d0466c9a14a57f839f6e5d55813889c22bc44b0c700f

5.5 Editing: remove, move, and cone removal

Three distinct operations, and conflating them loses data.

`remove( $x$ )` deletes a node and *contracts* the graph over the hole: x ’s children are reattached to x ’s parents, so nothing below x becomes unreachable. The result is again the unique minimal form.

`remove -cone( $x$ )` deletes x and *its contents*. In a multi-parent graph “its contents” needs a definition, and the only sound one is orphan collection: a member of the cone goes iff the root can no longer reach it once x is gone. So deleting a finished project removes the notes that belonged only to it and spares the specification another project still uses. Survivors are *detached*, never reattached upward — giving them the deleted node’s parents would assert something nobody said.

`move( $x$ , from  $P$ , to  $Q$ )` is the retracting twin of `put`: assert the new parents, retract the old. Everything below x travels with it for free, because membership is reachability rather than a stored subtree. A move can leave a shared child under both the old and the new category, which is true rather than broken — subsumption inherits, exclusive status cannot — so the tool reports those items instead of silently deciding.

5.6 Merging: the CRDT property

`merge( $D_1, D_2$ )` folds one graph into another: add every missing node, replay every edge through the same `put` machinery so that redundancies created by the union are pruned, and the result is again canonical.

Theorem 5.2 (Merge algebra; tested as invariant I7 in Földiák, 2026b). *merge is commutative and idempotent: merging A into B gives the same graph as merging B into A , and merging the same thing twice changes nothing.*

Those two properties, plus the canonical form, are the definition of a state-based conflict-free replicated data type (Shapiro et al., 2011). Their practical meaning:

Key idea

Any group of people can exchange changes **in any order, repeatedly, with no coordination**, and converge on the byte-identical graph. No server, no consensus protocol, no locking. Two writers who have folded in each other's work agree, and they can prove they agree by comparing 32 bytes.

Why does it work? Because the canonical state is defined as *the transitive reduction of the union of asserted edges*, and union is commutative and idempotent while reduction is a function. The correctness of this argument depends on reduction being *complete* — which is why the one-directional-redundancy bug mentioned in Section 5.2 was not cosmetic: with reduction incomplete, two writers could land on different roots for the same knowledge, and Theorem 5.2 was false.

The price is stated openly: merge is union, so it is *grow-only*. A removal can be resurrected by a peer who still holds the old edge. Deletion that must propagate needs either tombstones or a social protocol; ONTO DAG declines both and documents the wall.

5.7 Typed values: the order that is computed

Everything so far is asserted edges. But no finite set of edges can say that a parcel weighing 4 kg falls under “at most 5 kg”, because between any two weights there is another. So for *parametric* terms, the order is *computed from the name* instead of stored.

A term like `weight(4kg)` denotes a set of values; `weight(..5kg)` denotes the set of weights at most five kilograms; and one term is below another exactly when its denoted set is contained in the other's — the same “is a special case of” meaning every stored edge has. Values are exact rationals of the SI anchor unit, so every comparison is exact arithmetic and every replica computes the identical order.

```
$ odag prelude                                # declare the standard dimensions
$ odag put cello 'weight(4kg)'
$ odag put crate 'weight(25kg)'
$ odag put flute 'weight(400g)'

$ odag get 'weight(..5kg)'                    # everything under five kilos
cello
flute
weight(2/5kg)
weight(4kg)

$ odag canon 'weight(400g)'                   # what is actually stored
weight(2/5kg)

$ odag --render get 'weight(..5kg)'           # the same answer, read back nicely
cello
flute
weight(400g)
weight(4kg)
```

Three things in that transcript matter for the comparison with FCA.

First, `weight(..5kg)` is a *virtual* query term: no such node exists, and none is created. The query is answered by filtering the values that do exist. Constraints cost no writes.

Second, 400g is stored as `weight(2/5kg)` — canonicalised by *denotation*, not by spelling.

Two people who file the same weight in different units produce the identical stored form and hence the identical hash. This is the semantic canonicalisation of Section 5.4 extended to values.

Third, the round trip `--render` shows the readable spelling without ever storing it. The user’s phrasing is deliberately *not* kept: if it were, two people who know the same fact would disagree on the root.

Remark 5.3. Section 8.3 shows that this construction is an instance of a known FCA generalisation — interval pattern structures (Ganter and Kuznetsov, 2001; Kaytoue et al., 2011) — arrived at independently, and that the theory has things to say about it.

5.8 Persistence: from a graph to 32 bytes

ONTODAG’s persistence goes through `recordstore` (Földiák, 2026d), a small library providing a versioned key→record store over *content-addressed* storage — storage where a blob is retrieved by the hash of its own bytes and nothing is ever overwritten.

One record per node, keyed by name, holding its parents, children, count and optional payload. The index over those records is a canonically encoded radix trie, built so that equal contents always produce an equal index. So `commit()` returns one hash, the *root*, with three properties:

- the root names the entire dataset at that moment — share the root, share the dataset; keep an old root, keep a snapshot forever;
- equal content gives an equal root, always, so comparing two versions starts by comparing two strings;
- consecutive versions *share structure*: a small change writes only the changed records plus a thin path of index nodes.

Pointing the same machinery at a peer-to-peer content-addressed network (Ethereum Swarm; Trón, 2020) publishes the ontology into that network, with the latest root announced through a signed feed that others can follow. Section 11 is about what this buys.

5.9 Residency: eager, lazy, sparse

The same graph semantics come in three residencies, which matters for the thin-client story in Section 10. `EagerOntoDAG` loads every record into memory. `LazyOntoDAG` *reads* a published graph by fetching records only as a query walks them, so querying a published store costs the query rather than the store. `SparseOntoDAG` *writes* the same way: an edit touches only the records the reduction rules must check, and commits the byte-identical root a fully loaded editor would have produced. Published *cone summaries* — one derived record stating a broad category’s membership, in a separate store with its own root — turn broad queries from hundreds of fetches into a handful.

5.10 The guarantees

ONTODAG publishes a contract of what a higher layer may assume. Six clauses, each backed by a conformance test:

G1 Canonical root	Equal knowledge yields an equal root. The stored form is a <i>semantic</i> canonical form: build history, insertion order and the spelling of equal denotations do not affect it.
G2 Monotone merge	Merge is union then re-reduction. True <code>is_below</code> answers stay true; query results only grow. (Exception: a <code>remove</code> loses to a concurrent re-add.)
G3 Determinism	Same root and same interpretation context give the same answers on any replica and any residency.
G4 Fail-closed <code>is_below</code>	True only when the graph plus exact arithmetic <i>witness</i> it. False means “not derivable”, never “unknown but plausible”.
G5 Convergence	Writers who fold in each other’s published roots reach byte-identical roots regardless of gossip order.
G6 Overlap completeness	<code>get_overlapping</code> returns a recall-complete candidate set for a parametric term: complete for possibility, silent on satisfaction.

Notice how many of these are *negative* guarantees — promises about what will *not* happen (no history dependence, no shrinking answers, no plausible-but-underived *true*). Section 10 argues that negative guarantees are exactly what a machine consumer needs, and exactly what a language model cannot give about its own knowledge.

5.11 Summary of Section 5

Key idea

ONTODAG is **assertion kept minimal**. One graph, one root, no object/attribute distinction, one query primitive (intersection of cones), stored always in its unique transitive reduction. That uniqueness makes the graph canonical, canonicity makes it hashable, hashability makes it shareable and verifiable, and the union-plus-reduction definition of merge makes concurrent writers converge without coordination.

It answers the question “*what has been said, and what follows from it right now?*” — cheaply, incrementally, and in a form two strangers can compare in 32 bytes.

Part II — The correspondence

6 The dictionary: how the two formalisms correspond

We now have two objects built from the same six documents: Figure 5, a concept lattice with 10 elements, and Figure 7, an ONTODAG with 11 nodes and 17 edges. This section states exactly how they relate. The relationship is tighter than a family resemblance: there is a translation in both directions, and one of them is a classical theorem.

6.1 From an OntoDAG to a formal context

Given an ONTODAG whose node set is P , write $x \sqsubseteq y$ for “ x is below y ” — that is, x is in the descendant cone of y , so x is a special case of y . By Proposition 2.7 this is a partial order. Recall the two cones:

$$\downarrow y = \{x \in P : x \sqsubseteq y\} \quad (\text{descendants: the cone}), \quad \uparrow x = \{y \in P : x \sqsubseteq y\} \quad (\text{ancestors}).$$

Definition 6.1 (Ancestor-set code). For a node x , its *code* is its reflexive ancestor set, $\text{code}(x) = \uparrow x$. Read as a bit vector over the alphabet of all node names, $\text{code}(x)$ has a 1 for every category x falls under, including x itself.

This is the object Section 3 was talking about: a sparse binary codeword whose active bits are the categories the item belongs to. And the central property of a sparse code reappears as a triviality:

$$x \sqsubseteq y \iff \text{code}(y) \subseteq \text{code}(x).$$

More specific, more bits on. Subsumption is a subset test.

Definition 6.2 (The induced context). $\mathbb{K}(P) = (P, P, \sqsubseteq)$: objects are the nodes, attributes are *the same* nodes, and node x “has attribute” y exactly when $x \sqsubseteq y$.

The context is square and reflexive — an object is its own attribute. This is the formal shadow of ONTO DAG’s refusal to distinguish objects from attributes (§7.1).

Now compute the derivation operators of $\mathbb{K}(P)$ and watch what they turn into.

Proposition 6.3 (The dictionary). *In $\mathbb{K}(P)$, for $A \subseteq P$ (a set of nodes read as objects) and $B \subseteq P$ (read as attributes):*

$$A' = \bigcap_{x \in A} \text{code}(x), \quad B' = \bigcap_{y \in B} \downarrow y = \text{get}(B).$$

Proof. $A' = \{y : x \sqsubseteq y \text{ for all } x \in A\} = \bigcap_{x \in A} \{y : x \sqsubseteq y\} = \bigcap_{x \in A} \uparrow x$. Dually $B' = \{x : x \sqsubseteq y \text{ for all } y \in B\} = \bigcap_{y \in B} \downarrow y$, which is the definition of **get** from Section 5.3. $\square$

Key idea

FCA’s derivation operator on attribute sets is OntoDAG’s query. $B' = \text{get}(B)$, exactly, with no encoding or approximation. Dually, A' is “the categories all of these items share” — the intersection of their codes.

So the two formalisms are not analogous. They compute the same function. What differs is *when*: FCA computes it for every B in advance and stores the results; ONTO DAG computes it on demand and stores nothing.

That last sentence is, in one line, the entire thesis of this paper, and everything in Sections 7–11 elaborates it.

6.2 Formal concepts are self-describing queries

The dictionary makes the definition of a formal concept newly readable. (A, B) is a concept when $A' = B$ and $B' = A$, which now says:

A is the answer to the query B , and B is exactly the set of categories that every member of the answer shares.

A formal concept is a query *in balance* with its own answer — one that describes its result set precisely, with no category left out that the whole result happens to have. Most queries are not like this. Ask **get(Flight)** in the running example and you get three documents, all of which happen to be **Booked**; the query **Flight** is under-specified relative to its own answer. Its *closure* is $\{\text{Flight}, \text{Booked}\}$, and that closed set is the concept.

Try it yourself

Take the query $\{\text{Refundable}\}$ in Table 1. Its answer is **outbound**, **kyoto-inn**, **rail-pass**. What do all three share? **Refundable**, **Booked**, **Japan**. So the concept is number 5 in Table 2, and the implication $\text{Refundable} \rightarrow \text{Booked}, \text{Japan}$ of Section 4.7 is just this closure, written differently.

6.3 The theorem

Every node of the ONTODAG gives a concept, and the concepts that are not nodes are exactly the ones the ONTODAG was missing.

Proposition 6.4. *For every node $x \in P$, the pair $(\downarrow x, \uparrow x)$ is a formal concept of $\mathbb{K}(P)$, and $x \mapsto (\downarrow x, \uparrow x)$ is an order-embedding of P into $\mathfrak{B}(\mathbb{K}(P))$.*

Proof. $(\downarrow x)' = \bigcap_{z \sqsubseteq x} \uparrow z = \uparrow x$, since x itself is in $\downarrow x$ (giving $\subseteq$) and every upper bound of x is an upper bound of everything below x (giving $\supseteq$). Symmetrically $(\uparrow x)' = \downarrow x$. Injectivity is antisymmetry, and $x \sqsubseteq y \iff \downarrow x \subseteq \downarrow y$ gives the order-embedding. $\square$

Theorem 6.5 (ONTODAG $\rightarrow$ FCA; MacNeille, 1937; Ganter and Wille, 1999). *$\mathfrak{B}(\mathbb{K}(P))$ is the Dedekind–MacNeille completion of the poset P : the smallest complete lattice into which P embeds while preserving all meets and joins that already existed in P . It is unique up to isomorphism.*

Key idea

The concept lattice of an OntoDAG is its Dedekind–MacNeille completion. FCA does not describe a *different* structure from ONTODAG. It describes the *completion* of it: the same order, plus exactly the meets and joins needed to make every question of the form “what is the most specific thing under all of these?” have an answer inside the structure.

So the difference between the two formalisms is precisely measured by the elements the completion adds. Which we can count.

Measured, not asserted

For the running example’s ONTODAG (11 nodes plus the root):

17	concepts in $\mathfrak{B}(\mathbb{K}(P))$
11	of them are ONTODAG nodes (Prop. 6.4)
4	are genuine unnamed meets
2	are the $\top$ and $\perp$ a complete lattice must have

The four unnamed meets are “everything under Booked and Flight”, “... Booked and Japan”, “... Booked, Japan and Refundable”, and “... Booked, Flight and Japan”. Each is a real category. None was ever asserted.

6.4 From a formal context to an OntoDAG

The other direction is even simpler, and it is what Experiment 2 in the appendix actually does.

Definition 6.6 (Filing a context). Given $\mathbb{K} = (G, M, I)$, build an ONTODAG by putting each attribute $m \in M$ under the root, and each object $g \in G$ under $\{m : g I m\}$.

Proposition 6.7. *In the ONTODAG so built, for any $B \subseteq M$,*

$$\text{get}(B) \cap G = B'.$$

That is, the ONTODAG answers every FCA extent query exactly, in one intersection of cones, without ever computing a single concept.

This is worth pausing on, because it reframes the relationship in operational terms.

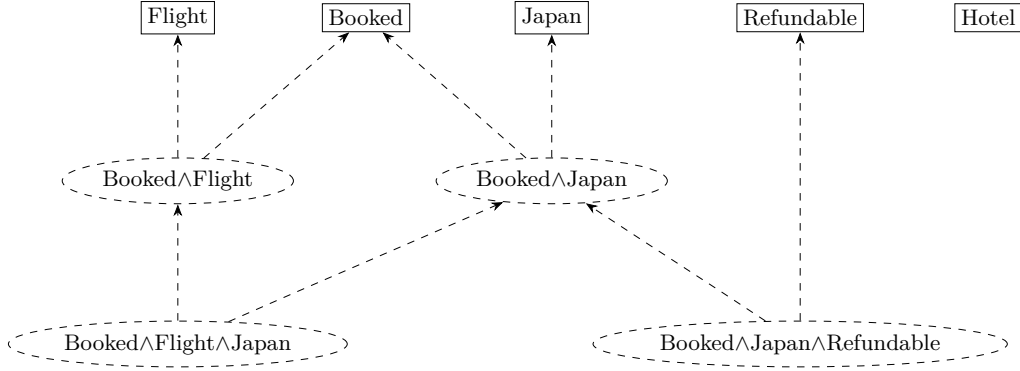


Figure 8: The completion at the category level, drawn as a Hasse diagram of intent containment. Solid boxes are the five categories the ONTODAG actually holds — all of them top-level, so no solid edges appear between them. Dashed ellipses are the four meets the Dedekind–MacNeille completion adds: combinations that some set of documents happens to share, but that nobody named. (The six documents are omitted; each hangs below every element whose properties it has, as in Figure 7.) FCA materialises all four — and, in larger data, exponentially many more; ONTODAG materialises none, and recomputes any one of them on demand as a cone intersection.

Key idea

OntoDAG is FCA with the closure computed lazily. The concept lattice is a *memo table*: it precomputes the extent of every closed query. ONTODAG stores the raw incidence and computes extents per query. The two give identical answers; they differ in what is paid for in advance.

That framing also says what the middle ground looks like: materialise *some* concepts — the ones that are queried often enough, or that compress the data enough, to pay for themselves. Sections 8.5 and 12.3 pursue this.

6.5 Codes and graphs are the same information

One more correspondence, this time internal to ONTODAG, because it closes the circle back to Section 3.

Proposition 6.8. *The family of codes $\{\text{code}(x)\}_{x \in P}$ determines the ONTODAG completely: the covering relation of containment among the codes is exactly the stored transitive reduction. The maps*

$$\text{DAG} \xrightarrow{\text{reflexive ancestor closure}} \text{codes}, \quad \text{codes} \xrightarrow{\text{covering relation of } \subseteq} \text{DAG}$$

are mutually inverse.

Measured, not asserted

Tested on 200 randomly generated ONTODAGs of 4–12 nodes: in **200/200** cases, the covering relation reconstructed from the codes alone equalled the stored edge set, edge for edge.

So the codeword picture is not a lossy intuition; it is a faithful alternative representation. A neural code of the right shape *is* an ONTODAG, and an ONTODAG *is* a neural code. That is the mind–brain bridge of Figure 4, made concrete and checked by execution.

Remark 6.9 (Why the code cannot be the stored form). Being equivalent does not make it equally good to store. A code is a set of names, so storing codes stores the transitive *closure* —

quadratically more edges, and every insertion touches every descendant. Worse, if you compress the code into a bit vector you must fix an assignment of names to bit positions, and that assignment is a global, history-dependent decision — the enemy of canonical form. ONTODAG therefore stores the reduction (unique, local, canonical) and treats codes and cone bitmaps as a *derived, regenerable index*, in the same category as the descendant counts.

6.6 The complete dictionary

Table 3: The translation table. Rows above the rule are exact correspondences; rows below are places where one side has no counterpart — these are the subject of Section 7.

FCA	OntoDAG	Both are
formal context (G, M, I)	the asserted graph	a binary incidence / a code
object g	a node	a thing with a name
attribute m	a node	<i>same thing</i> : no split
intent B of a concept	$\text{code}(x)$, the ancestor set	the codeword
extent A of a concept	$\downarrow x$, the cone	the members
B' (attribute derivation)	$\text{get}(B)$	intersection of cones
A' (object derivation)	shared categories	intersection of codes
closure B''	the query’s own description	a closure operator
$(A_1, B_1) \leq (A_2, B_2)$	$x \sqsubseteq y$	containment
meet $\wedge$	get of two terms	intersection of extents
concept lattice $\mathfrak{B}(\mathbb{K})$	the DM completion of the graph	Thm 6.5
implication $B \rightarrow C$	(no stored counterpart)	$C \subseteq B''$
has no OntoDAG counterpart		
completeness of the lattice	—	§7.2
Duquenne–Guigues base	—	§8.1
attribute exploration	—	§8.6
has no FCA counterpart		
—	stable node names	§7.4
—	transitive reduction as storage	§7.3
—	merge of two structures	§7.5
—	computed (infinite) value order	§8.3
—	content-addressed root	§11

6.7 Where the analogy leaks: the meet-substitution trap

The dictionary is exact, but there is one place where a careless reader will draw a false conclusion, and it is worth a boxed warning because it has real consequences for optimisation.

In a concept lattice, the meet of two concepts always exists. It is tempting to think that an ONTODAG node placed under both A and B therefore *is* the meet of A and B — so that a query planner could answer $\text{get}(A, B)$ by looking up that node’s cone. It cannot.

Measured, not asserted

Put AB under $\{A, B\}$; then put X under $\{A, B\}$ as well. Measured cones:

$$\downarrow A \cap \downarrow B = \{AB, X\}, \quad \downarrow AB = \emptyset.$$

Watch out

A node under A and B is a **sibling** of the meet, never the meet. Substituting it for the pair $\{A, B\}$ in a query plan silently loses results. This is not a bug: it follows from ONTODAG storing assertions, since nobody said that X is a kind of AB . The corresponding FCA concept — the real meet — exists only in the completion (Figure 8), where it is one of the dashed ellipses.

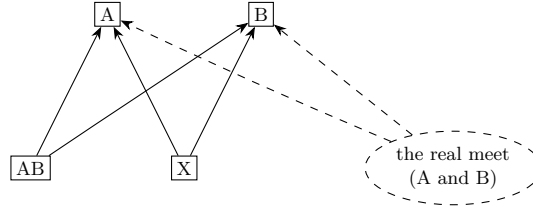


Figure 9: AB and X are both under A and B , so neither is above the other, and neither is the meet. The meet (dashed) is the concept whose extent is $\{AB, X\}$; it exists in the completion and has no node in the graph.

The general lesson generalises well beyond query planning, and Section 10 leans on it: **OntoDAG has no defined concepts**. Every node means exactly what was asserted about it, never what could be inferred to fill a gap. A system that wants defined concepts — “Refundable Japan Booking *means* whatever is under all three” — must either assert the placements or compute the meet at query time. It will not get one by accident.

7 The differences that matter

Section 6 showed how much is shared. This section isolates what is not, and argues that the divergences are not accidents of implementation but a coherent set of trade-offs, each one bought with a specific currency.

There are six. The first four are deep; the last two are consequences.

7.1 Difference 1: one sort, not two

FCA is built on a strict separation. G and M are different sets; an object is never an attribute; the whole apparatus of primes depends on knowing which side of the table you are on. ONTODAG has one sort: nodes, all the way down.

The separation buys FCA something real — the Galois connection is a statement about *two* sets, and the duality it creates is elegant and generative (every theorem comes with a mirror image). What it costs is awkwardness in exactly the place ontologies live.

Where it hurts: attributes have no hierarchy. In Table 1 there is no way to say that Flight is a kind of Travel. Flight is an attribute; Travel would have to be another attribute; and nothing in the formalism relates two attributes. The standard workarounds all move the problem rather than solve it:

- add Travel as a redundant column and mark it wherever Flight or Hotel is marked — the hierarchy is now encoded but not *stated*, and nothing prevents an inconsistent row;
- use conceptual scaling, which is really the same trick with a name and a discipline;
- move to a many-sorted extension such as Relational Concept Analysis (Rouane-Hacene et al., 2013), which is powerful but is a substantially larger formalism with an iterative fixpoint computation.

ONTODAG gets attribute hierarchies for nothing, because `Flight` is a node and `put Flight Travel` is the same operation as `put outbound.pdf Flight`. The corollary — and it is a real consequence, not a slogan — is that *the same query primitive traverses both levels*: `get(Travel)` returns `Flight`, `Hotel` and all the documents beneath them, without a join, a second index, or a special rule.

What it costs OntoDAG. No duality. There is no “dual query” in ONTODAG corresponding to A' — “the categories all these items share” is computable but is not a first-class operation with its own optimiser, and no theorem comes for free in mirror image. More importantly, ONTODAG cannot distinguish, in its data model, between “this thing is an individual” and “this thing is a class”. Whether `outbound.pdf` may acquire children is a social question, not a checkable one. FCA users get that check free.

Key idea

The one-sorted design is ONTODAG’s largest departure from FCA and its most consequential. It buys hierarchies of categories with no extra machinery, at the cost of the object/attribute duality and of any built-in class/individual discipline.

7.2 Difference 2: asserted, not closed

FCA materialises every concept the data supports. ONTODAG materialises none beyond what was said. Theorem 6.5 says the difference is exactly the Dedekind–MacNeille completion; Section 4.9 says what the completion costs.

The argument for closing is strong and should be stated fairly. Closure *finds things*. The four unnamed meets of the running example are real categories that nobody thought to write down, and in a scientific dataset those are often the point — that is the entire premise of Endres and Földiák (2008)’s use of FCA on neural data.

The arguments against closing, for a system with ONTODAG’s job description, are four:

1. **Size.** Exponential in the worst case, and the worst case is a perfectly ordinary table (§4.9). A store that must fit in a browser tab and be hashed after every edit cannot have a representation whose size is #P-hard to predict.
2. **Coincidence.** FCA creates a concept for any co-occurrence, including accidental ones. With six documents, “booked flights to Japan” is meaningful; with six thousand, most closed sets are noise. FCA has no notion of a concept not being worth having — which is precisely the observation that motivates iceberg lattices, stability, and the MDL programme of Section 8.5.
3. **Instability under edit.** Adding one object can create, destroy and rearrange many concepts. For a batch analysis this is fine. For a store that is edited continuously and hashed after every edit, it means the representation churns globally on local changes — which destroys structural sharing, and with it the cheap snapshots of Section 5.8.
4. **Authorship.** A derived concept has no author. In a multi-writer setting, “who said this?” must have an answer (Section 10); a concept that exists because of a coincidence between two people’s unrelated contributions has no honest answer to that question.

Key idea

ONTODAG’s position: the closure is real and often valuable, but it is *derived* — so compute it on demand, locally, per query, and never store it in the shared, hashed, merged graph. This is the same discipline ONTODAG applies to every derived structure it has (descendant counts, cone summaries, indexes): regenerable, local, evictable, outside the canonical root.

7.3 Difference 3: two kinds of canonical form

Both formalisms have a uniqueness theorem, and comparing them is the most precise way to see what ONTODAG is optimising for.

	FCA	OntoDAG
canonical object	the concept lattice $\mathfrak{B}(\mathbb{K})$, unique up to isomorphism (Thm 4.7); and the Duquenne–Guigues base, unique on the nose (Guigues and Duquenne, 1986)	the transitive reduction, unique on the nose (Thm 2.11)
what it quotients away	nothing — it is a complete description	assertion order, redundant edges, spelling of equal values
cost to compute	exponential in the worst case; #P-complete to size (Kuznetsov, 2001)	incremental and local: one edge plus its ancestor set
cost to maintain	recompute (a new object may restructure everything)	delta on the affected region
usable as an address?	in principle; in practice too big and too unstable	yes: hash the serialisation, 32 bytes

Both are canonical forms. Only one is a canonical form you can *use as an identifier*, and that is the whole difference.

Key idea

OntoDAG chose the cheapest non-trivial canonical form that is still semantic. It is semantic — not merely syntactic — because it quotients away things that do not change meaning: the order you typed facts in, links you typed that were already implied, and the way you spelled a value (400 g versus 2/5 kg). It is cheap because the quotient is computed by local edge deletion rather than global inference.

Every step up the expressiveness ladder threatens this. Add arbitrary logical axioms and canonicalising “up to logical equivalence” means canonicalising an entailment closure; the root silently degrades from a fingerprint of *knowledge* to a fingerprint of *phrasing*. That, rather than tractability, is the real wall ONTODAG is standing behind.

7.4 Difference 4: concepts with names, and concepts without

This is the subtlest difference and, for the applications in Part IV, the most important.

In FCA, a concept *is* its (extent, intent) pair. It has no other identity. This has a consequence that is easy to miss: **concepts do not survive changes to the data.** Add one document to Table 1 and the concept ($\{\text{outbound, inbound}\}, \{\text{Flight, Japan, Booked}\}$) may cease to exist — not change, cease. If the new document is a third booked Japanese flight, the pair with that extent is simply not a concept of the new context. Something very like it exists, with a different extent. Whether it is “the same concept” is a question the formalism cannot answer, because it has no notion of sameness across contexts.

In ONTODAG, a node *is* its name. `Flight` is `Flight` before and after any edit, in your copy and in mine, in this version and in the version from last year. Names are the identity at every boundary — the public API, the serialisation, and anything crossing between two graphs.

Four things follow, and each of them is load-bearing later:

1. **Merge is possible at all.** Two graphs can be merged because “Flight here” and “Flight there” are the same thing by definition. Two concept lattices over different contexts have no such correspondence; there is no canonical way to merge them.
2. **References are stable.** An external system can hold a reference to `Flight` and it will still mean something next year. A reference to a concept-as-a-pair is invalidated by the next edit.

3. **Provenance is possible.** “Alice asserted that `outbound.pdf` is under Japan” is a durable statement about a durable subject. “Alice asserted concept #7” is not.
4. **The names must be agreed.** This is the price. ONTODAG’s correctness depends on people using the same string for the same idea, and it has no mechanism to enforce that. FCA, deriving everything from data, never has to ask.

Key idea

FCA derives concepts and therefore cannot name them stably. ONTODAG names concepts and therefore cannot derive them. This is a genuine trade, not an oversight on either side, and it is the reason the two formalisms end up serving different masters: FCA serves the analyst looking at a fixed dataset; ONTODAG serves the collaborator, the archive and the verifier, all of whom need to point at the same thing twice.

There is a long historical argument on ONTODAG’s side of this trade, which Section 9.2 develops: every attempt to make identifiers carry their own meaning — Wilkins’s philosophical language, Dewey decimal classification, Linnaean binomials — has paid for it in instability when the classification was revised, and the mature practice in every field that tried (chemistry is the cleanest case) is to keep *both* an arbitrary stable key and a derived meaningful code.

7.5 Difference 5: a fixed table, or a living store

FCA analyses a context. The context is an input; it does not change while the analysis runs; there is one of it.

ONTODAG is a store. It is edited continuously, by several people who may never communicate, each of whom holds a replica; and it must converge. Almost every design decision above is downstream of that requirement:

Requirement	What it forced
edits arrive one at a time	incremental reduction, delta-maintained counts
replicas must converge	merge = union + re-reduction: commutative and idempotent (Thm 5.2)
agreement must be checkable	canonical form $\Rightarrow$ one hash (§5.4)
answers must not shrink	monotone semantics: no negation in the shared store (G2)
old answers must stay valid	immutable roots; non-monotone questions must name one (§10.4)

Note the third row in particular. In a merging system, *monotonicity* is not a stylistic preference; it is what makes union a sound merge. A query whose answer can shrink when new information arrives cannot be answered consistently by two replicas that have seen different subsets of the news. This is why ONTODAG refuses negation, closed-world readings and aggregation in the living store — and it is also why it can offer them safely against a pinned root, where the world is closed by *scoping* rather than by decree.

FCA has no analogue of any of this, and needs none. But it also means there is no such thing as “merging two concept lattices”, and that a distributed, multi-author FCA is not a thing you can build by adding features to FCA. You would build it by doing what ONTODAG does — sharing the *assertions* and letting each replica close locally.

7.6 Difference 6: finite crosses, or computed order

An FCA context is a finite table of crosses. ONTODAG additionally supports an order that is *computed from names*: `weight(4kg)` is below `weight(.5kg)` because $4 \leq 5$, with no edge stored and no node for the query term (§5.7).

In FCA terms, the closest analogue is conceptual scaling — turn each threshold into an attribute. But scaling must choose the thresholds in advance, which means quantisation error and

a fixed granularity, and it makes the context grow with the number of thresholds. ONTODAG's values are exact rationals, arbitrary thresholds are free, and the shared structure carries only the values that are actually used.

The real FCA analogue is *pattern structures* (Ganter and Kuznetsov, 2001; Kaytoue et al., 2011), which ONTODAG appears to have reinvented in one special case. Section 8.3 takes this up, because it is the clearest example in the paper of theory that ONTODAG could usefully import.

7.7 Summary: the two systems side by side

Table 4: The comparison in one table. A reader who skipped Parts I and II can start here.

	Formal Concept Analysis	OntoDAG
Input	a fixed binary table	a stream of assertions
Sorts	two: objects, attributes	one: nodes
Output	every concept, as a complete lattice	the asserted order, minimally stored
Size	up to exponential	linear in what was said
Meets	always exist	only if someone asserted one
Concept identity	the (extent, intent) pair	a name string
Survives an edit?	concepts may cease to exist	names persist
Canonical form	the lattice; the stem base	the transitive reduction
Canonical form usable as an address?	no (too large, too volatile)	yes: 32-byte root
Merging two of them	undefined	commutative, idempotent, convergent
Query	lattice navigation / lookup	one cone intersection, planned
Numeric values	conceptual scaling, or pattern structures	exact rationals, order computed from names
Negation	expressible on the fixed table	refused in the living store; allowed against a pinned root
Provenance	no place for it	signed records, per claim
Verification	re-run the algorithm	Merkle proofs and certificates
Learns concepts from data	yes, all of them	no — deliberately
Best at	<i>discovering</i> structure in a dataset	<i>agreeing on</i> structure between parties

The last row is the honest one-line summary. FCA is a discovery instrument. ONTODAG is an agreement instrument. Almost every difference above follows from that, and Part III is about what each can take from the other without becoming the other.

Part III — What each can learn from the other

8 What OntoDAG can learn from FCA

FCA has forty years of theory; ONTODAG has a working system and a short list of invariants it will not give up. This section goes through the parts of the theory that could be imported, and for each asks four questions:

- (a) what is it, in one paragraph a first-year reader can follow;
- (b) what would ONTODAG gain;

- (c) what would it cost — specifically, which of ONTODAG’s guarantees (§5.10) does it threaten;
- (d) verdict.

The verdicts use a fixed vocabulary, because the distinction between them is the most transferable thing in this section:

Adopt (core)	belongs in the shared, hashed, merged graph
Adopt (derived)	belongs in a separate local index: regenerable, evictable, never merged, outside the canonical root
Adopt (surface)	belongs in the interface layer — rendering, suggestion, tooling — and touches no stored bytes
Research	desirable, but breaks something until a real problem is solved
Refuse	incompatible with an invariant ONTODAG exists to provide

That vocabulary is itself a lesson learned from FCA: the reason ONTODAG can consider adopting so much is that it has a place to put derived things where they cannot contaminate the canonical form.

8.1 Implications and the stem base

(a) What it is. An implication $B \rightarrow C$ holds when everything having all of B also has all of C (§4.7). The Duquenne–Guigues *stem base* (Guigues and Duquenne, 1986) is the unique smallest set of implications generating all the others. It can be computed by NextClosure at the same time as the concepts (Ganter, 2010).

(b) What OntoDAG would gain. Two genuinely useful things.

First, a **suggestion engine** for classification — which is exactly the gap ONTODAG’s contract admits it has (“classification is the higher layer’s job”). Measured on the running example, FCA reports $\text{Refundable} \rightarrow \text{Booked, Japan}$. If a user now files a refundable document *without* marking it **Booked**, the system can say: *every other refundable thing here is booked — did you mean to mark this one?* That is a well-founded prompt, computed from the store’s own contents, needing no language model and no external ontology.

Second, a **redundancy lint**. If $\text{Hotel} \rightarrow \text{Travel}$ holds extensionally for every hotel in the store, someone may have forgotten to assert the edge $\text{Hotel} \rightarrow \text{Travel}$ in the graph. The implication is evidence for a missing subsumption.

(c) What it would cost. Storing implications in the shared graph would be a category error, and it is worth being precise about why, because the same argument recurs throughout this section.

An extensional implication is a **fact about today’s population, not a law**. $\text{Flight} \rightarrow \text{Booked}$ holds in the running example only because nobody has yet filed an unbooked flight. Now consider what happens under merge: a collaborator who has one unbooked flight merges their store into yours, and the implication is destroyed. An implication is therefore *non-monotone* — it can be falsified by the arrival of new information — which is exactly the property that guarantee G2 forbids in the shared store. Stored as data, implications would make two replicas disagree, and would make the canonical root depend on which subset of the world a replica had seen.

There is a second, subtler cost: computing the stem base is expensive (equivalent in the worst case to enumerating pseudo-intents, which is not polynomial-delay), so it is a batch job, not something to run on every write.

(d) **Verdict: Adopt (derived) for the lint, and Adopt (surface) for the suggestion.** Compute implications in a separate, local, regenerable index — the same discipline ONTODAG already uses for cone summaries — and surface them as *prompts to a human or an agent*, never as stored claims. If the user accepts a prompt, what gets stored is an ordinary asserted edge with ordinary provenance. The implication was evidence; the edge is the claim. Keeping that distinction is the difference between a helpful system and one that silently invents knowledge.

Key idea

The general rule this instance establishes: **FCA’s derived output may enter OntoDAG only as a suggestion to an author, never as an assertion by the system.** Everything that follows in this section obeys it.

8.2 Conceptual scaling, and the levels of measurement

(a) **What it is.** Conceptual scaling (§4.10) turns non-binary data into a context by choosing, per attribute, which comparisons matter: nominal (distinct values), ordinal (thresholds), interordinal (both ends), and so on (Ganter and Wille, 1999, ch. 1).

(b) **What OntoDAG would gain.** Not a mechanism — ONTODAG already has one — but a **vocabulary and a completeness check**. ONTODAG’s dimension kinds turn out to line up exactly with a classification that predates both systems: Stevens’s levels of measurement (Stevens, 1946). Canonicalisation, in each case, is quotienting by the transformations the scale considers meaningless:

Stevens level	Meaningful transformations	FCA scale	OntoDAG kind
nominal	any relabelling	nominal	plain nodes
ordinal	any order-preserving map	ordinal	<i>missing</i>
interval	affine $x \mapsto ax + b$	interordinal	affine (temperatures)
ratio	scaling $x \mapsto ax$	interordinal	linear (weight, length)
absolute	none	nominal	count

The table has one conspicuous hole, and finding holes is what a good vocabulary is for: ONTODAG has *no ordinal kind*. There is no way to say that **beginner** < **intermediate** < **expert** as a declared dimension, so such chains have to be built as ordinary edges, which works but is unindexed and gives no arithmetic. FCA’s ordinal scale is the template for what the declaration would look like.

(c) **What it would cost.** Adding an ordinal kind is cheap on both admissibility axes: it is monotone, and the order is computable from the declared chain plus the value names, so canonicity is preserved. The real risk is social rather than technical — an ordinal kind invites users to fake arithmetic on ranks (“the average of beginner and expert”), which is exactly the abuse Stevens wrote his paper to warn against.

(d) **Verdict: Adopt (core), carefully.** The scale vocabulary should be documented as ONTODAG’s own; the ordinal kind should be added, with the arithmetic deliberately absent.

8.3 Pattern structures: the theory OntoDAG already uses

(a) **What it is.** Ganter and Kuznetsov (2001) generalise FCA so that an object’s description need not be a set of attributes. Instead descriptions live in a meet-semilattice $(D, \sqcap)$ with a

similarity operation. The derivation operators become

$$A^\square = \prod_{g \in A} \delta(g), \quad d^\square = \{g \in G : d \sqsubseteq \delta(g)\},$$

and the whole theory — concepts, lattice, implications — goes through unchanged. The most-used instance is the *interval pattern structure*: descriptions are intervals, $\sqcap$ is the convex hull, and $d \sqsubseteq e$ means $e \subseteq d$ as intervals (Kaytoue et al., 2011).

(b) What OntoDAG would gain. Recognition, and then results. ONTODAG’s parametric dimensions (§5.7) *are* an interval pattern structure: `weight(..5kg)` is an interval, containment of denoted value sets is $\sqsubseteq$, and the star of values under a dimension head is the set of descriptions in play. ONTODAG arrived at this independently, from the practical need to answer a courier’s “anything under five kilos?” without pre-quantising.

Knowing that it is an instance of a studied formalism buys three things:

- **Correctness assurance.** The pattern-structure framework tells you exactly what conditions $\sqcap$ must satisfy for the theory to hold (associativity, commutativity, idempotence — a semilattice). ONTODAG’s requirement that only exact-arithmetic kinds may enter the canonical order is the same condition, discovered the hard way.
- **Projections.** Pattern structures come with a theory of *projections*: simplifying descriptions (coarsening intervals) in a way that provably yields a sound approximation of the concept lattice. This is the principled version of the “derived index for hot dimensions” idea that ONTODAG currently treats as an engineering hunch.
- **Multi-dimensional descriptions.** Kaytoue et al. (2011) handle a *vector* of intervals as one description — which is precisely ONTODAG’s open question about how several coordinates (`from`, `to`, `duration`) should combine. The pattern-structure answer is the product semilattice with componentwise $\sqcap$, and it comes with the meet already defined.

(c) What it would cost. Nothing, if imported as *theory*. Importing the *machinery* — computing the pattern concept lattice — would reintroduce the closure problem of Section 7.2, with the added hazard that interval pattern lattices are typically larger than their binary counterparts.

(d) Verdict: Adopt (theory), immediately and at no cost. Re-describe ONTODAG’s dimensions in pattern-structure language in the design documents; use the projection theory when the per-dimension index is finally built; adopt the product-semilattice answer for multi-coordinate descriptions when that question is decided. This is the single highest-value, lowest-cost item on the list: it is free, and it converts an engineering hunch into a known-correct construction.

8.4 Stability, support, and knowing which concepts are worth having

(a) What it is. Two ways of ranking concepts. *Support* is extent size — keep the concepts with many members, which yields the iceberg lattice (Stumme et al., 2002). *Stability* (Kuznetsov, 2007) is the fraction of sub-collections of the extent that would still produce the same intent — keep the concepts that do not depend on any one or two objects.

(b) What OntoDAG would gain. Half of this machinery already exists. The stored `descendant_count` *is* the extent size of a node, kept current by delta on every write. So half of this machinery exists and is already load-bearing (the query planner uses it to order cones). What is missing is the *use* of these numbers to decide what deserves to exist.

Concretely, three consumers:

1. **Which unnamed meets to suggest.** Of the four meets in Figure 8, which should the system offer to name? Support and stability rank them. From Table 2, “booked Japan documents” has support 4 but stability 0.38; “refundable Japan bookings” has lower support (3) but higher stability (0.62). The second is the better candidate for a category, and only stability says so.
2. **Which cones to index.** ONTODAG’s published cone summaries currently use a fixed threshold rule on `descendant_count`. That is iceberg selection under a different name, and the FCA literature has better selection criteria.
3. **Which concepts to materialise in a query planner.** The topic of Section 12.3.

(c) **What it would cost.** Stability is expensive to compute exactly (it is a sum over subsets; the experiment script brute-forces it only for extents of at most 16). Approximations exist and are standard practice. Since the output is a ranking used for suggestion and indexing, approximation is harmless — it steers effort, never correctness.

(d) **Verdict: Adopt (derived).** Support is already there; add estimated stability to the derived index, and use both to rank suggestions. Never let either affect the stored graph.

8.5 Boolean factor analysis and the MDL connection

(a) **What it is.** Bělohlávek and Vychodil (2010) proved a striking result: if you want to decompose a binary matrix into a product of two binary matrices with the fewest inner dimensions — Boolean factor analysis — then the optimal factors are *formal concepts*. Concepts are not just an interpretive device; they are the right basis.

That result immediately raises the model-selection question, because using *all* concepts is the trivial and useless decomposition. Which subset? The minimum description length principle (Rissanen, 1978; Grünwald, 2007) gives a principled answer: keep a concept if describing the data using it is shorter than describing the data without it — if it *pays for its own description*. This is the programme of Miettinen and Vreeken (2014) and, in the DAG-structured form relevant here, of the `mdl-fca` project (Földiák, 2026a). The same two-part-MDL move drives the pattern-set miners Krimp and Slim (Vreeken et al., 2011; Smets and Vreeken, 2012), which select a small set of itemsets that compress the data; what a DAG-structured version adds is that a new concept sits *above* its constituents, and can make earlier ones redundant.

(b) **What OntoDAG would gain.** A principled answer to the question ONTODAG currently punts on: *where do categories come from?* Today they come from people. MDL-selected concepts would let a store propose new categories that are justified by compression — Barlow’s “suspicious coincidences” (Barlow, 1989) made exact.

This is also the cleanest resolution of the tension in Section 7.2. The complaint against closure was that FCA creates a concept for every coincidence. MDL is precisely a filter for coincidence: a concept that arises from chance co-occurrence does not compress, so it does not pay, so it is not admitted. *MDL-FCA is the principled version of iceberg partiality* — support thresholds ask “how many?”, MDL asks “does it earn its keep?”.

(c) **What it would cost.** The learned structure is *global and history-dependent*: it is a function of the whole dataset, and it changes when the data changes. Merging two learned DAGs would be merging two models fitted to different samples, which is not union and does not converge. Putting learned concepts directly into the shared graph would break G1 and G5 at once.

(d) **Verdict: Adopt (derived), with a clearly specified promotion path.** The layering that makes it safe already exists:

- run the learner over a *pinned root* — so its input is immutable, its output is reproducible, and both can be cited;
- keep the learned structure in a separate store with its own root, never merged (the same pattern as cone summaries);
- if a learned category is to become shared knowledge, it enters as an *ordinary asserted claim by an identified author*, with the pinned root it was derived from recorded as its basis. Someone — a person or an accountable agent — takes responsibility for it, and it becomes an assertion like any other.

Notice that this preserves the discipline of Section 8.1 exactly: the learner produces evidence, an author produces claims.

8.6 Attribute exploration, with a language model as the expert

This is the most interesting item on the list, because a forty-year-old protocol has just acquired a participant it was always waiting for.

(a) **What it is.** Attribute exploration (Ganter and Obiedkov, 2016) is an interactive knowledge-acquisition loop:

1. The system computes the next implication that its current data supports and that does not already follow from what has been confirmed.
2. It asks a domain expert: *is this always true?*
3. If the expert says yes, the implication is added to the knowledge base.
4. If the expert says no, they must supply a **counterexample** — an object violating it — which is added to the context.
5. Repeat until no unconfirmed implications remain.

On termination you have an implication base that is complete for the *domain*, not merely for the sample — which is a strictly stronger thing than data mining can give you, and the reason the protocol was invented.

(b) **What OntoDAG would gain.** A principled, terminating, *auditable* procedure for growing an ontology, in which the expensive step — answering domain questions — is exactly the step a modern language model is good at, and in which every answer is a discrete, recordable, attributable commitment.

Consider how well the pieces fit:

What exploration needs	What OntoDAG has
a context to mine implications from	the asserted graph, viewed as $\mathbb{K}(P)$ (§6)
an expert who answers questions	an agent on the MCP surface (§10)
counterexamples added as objects	put — the ordinary write path
a record of what was confirmed	signed provenance records, per claim
a way to revisit a bad answer	retraction records; roots to compare against
termination	the base is finite

And it addresses the central problem of agent-written knowledge, which Section 10.8 states as: *agents write faster than anyone can review*. Exploration inverts the flow. Instead of an agent proposing arbitrary edits for review, the *system* generates the questions — a minimal, non-redundant sequence, each one maximally informative given the answers so far — and the agent answers them. Review becomes bounded, ordered and non-redundant, which is exactly what makes oversight tractable.

(c) What it would cost. Three real problems, none fatal.

First, **the expert can be wrong**, and a language model can be confidently wrong. Classical exploration assumes an infallible expert. The mitigation is already ONTODAG’s design: an answer is not truth but a *signed claim*, so a wrong answer is attributable, disputable and retractable, and readers may weigh authors differently. The protocol degrades from “complete for the domain” to “complete relative to what this author asserted”, which is the honest thing for it to be.

Second, **implications are not subsumptions**. Attribute exploration yields $B \rightarrow C$ for sets B , and ONTODAG stores only single-parent edges $x \sqsubseteq y$. A confirmed implication with a compound premise ($\{\text{Flight, Japan}\} \rightarrow \text{Booked}$) has no representation in the core graph. This is the relations/axioms wall, and it is where the honest answer is: store the implication in the provenance layer as a claim, use it as a lint, and do not pretend the core entails it.

Third, **exploration assumes a fixed universe**, while ONTODAG stores merge concurrently. Two agents exploring two replicas will ask different questions and confirm different things. Reconciling that is genuinely open.

(d) Verdict: Research, and the most promising item in this paper. The single-writer case — one store, one agent, one human reviewing the confirmed implications — is buildable today with existing pieces, and would be a genuinely novel artefact: *an ontology grown by a language model under a protocol that bounds and orders the review*. The multi-writer case needs the concurrent-exploration question answered first. Section 12.3 sketches the design.

8.7 Drawing: line diagrams as a design discipline

(a) What it is. FCA takes diagram layout seriously, because the whole point of Wille’s programme was that a domain expert should be able to *read* a lattice (Wille, 1982). The techniques — additive layouts where each attribute contributes a fixed vector, reduced labelling (§4.4), and layer assignment by concept rank — are mature.

(b) What OntoDAG would gain. Better pictures, and one specific improvement worth naming. ONTODAG’s visualiser draws the graph with Graphviz, labelling every node with its full name. For a store with typed values that is noisy: fifty weight values under one head, each drawn as a box. Reduced labelling is the direct fix — draw the structure, and place each name once, where it is introduced. Compare Figure 5 with Figure 7: the former carries the same information in less space, and the difference is entirely the labelling convention.

(c) What it would cost. Nothing; it is a rendering choice.

(d) Verdict: Adopt (surface). Cheap, and the reading rules would have to be documented, which is a small cost against a real gain in legibility.

8.8 Algorithms and complexity as guardrails

(a) What it is. FCA has decades of algorithm engineering (Kuznetsov and Obiedkov, 2002; Krajca et al., 2010; Lindig, 2000) and a clear complexity picture (Kuznetsov, 2001).

(b) What OntoDAG would gain. Chiefly, **knowing where the cliffs are**. The contranominal scale (§4.8) is a small, memorable object whose lattice is 2^n ; recognising that pattern in a proposal is a fast way to reject it. NextClosure’s constant-memory, lexic-order enumeration is the right algorithm for any future ONTODAG feature that must walk closed sets without materialising them — an implication lint, for instance, or an MDL learner’s candidate generation.

There is a second, subtler gift. FCA’s complexity results explain *why* ONTODAG’s design is not merely a convenient shortcut. A system that promised “all concepts” would be promising something #P-hard to even count. ONTODAG’s refusal is principled, and the citation for that principle is Kuznetsov (2001).

(d) Verdict: Adopt (theory). Free.

8.9 Triadic FCA, and provenance

(a) What it is. Lehmann and Wille (1995) extend contexts to three dimensions: objects $\times$ attributes $\times$ *conditions*, with triadic concepts as triples.

(b) The tempting analogy. ONTODAG’s provenance layer records *who* asserted *which* claim: a third axis over the same claims. “Claims that Alice and Bob both endorse” looks like a triadic concept.

(c) Why it does not fit as-is. Triadic FCA treats the third dimension symmetrically with the other two, and ONTODAG’s does not: authorship is not a property of knowledge but of a speech act, deliberately stored in a *separate* store with its own root so that the knowledge root stays a fingerprint of knowledge alone. Merging the axes would put “who said it” inside the hash of “what is true”, which is precisely the design error the separation exists to prevent — two people who know the same thing must get the same root.

(d) Verdict: Adopt (theory), refuse (core). The triadic *analysis* could run over the provenance store as a derived view — “which authors agree about which regions of the ontology” is a real and interesting question, and triadic concepts are the right shape for it. The triadic *representation* must stay out of the knowledge graph.

8.10 What OntoDAG should refuse

Three items, for completeness.

Fuzzy and graded FCA (Bělohlávek, 2004). **Refuse.** A graded incidence puts a number in the data. Numbers in identity break exact canonicalisation (which number, at what precision, rounded how?) and break merge (whose number wins?). Confidence is a property of a *speech act*, and belongs on a provenance record where it can be attributed and where disagreement is representable as disagreement rather than as a silently averaged scalar.

Full closure in the shared store. **Refuse**, for the four reasons of Section 7.2. Local closure over a pinned root is fine and is the recommended way to get the benefits.

Relational Concept Analysis as the relations answer (Rouane-Hacene et al., 2013). **Research.** ONTODAG genuinely lacks relations: “Alice *authored* Doc1” is not subsumption, and the system has no way to say it. RCA is the mature FCA-native answer, handling inter-object links by iterated relational scaling to a fixpoint. But the fixpoint is a global computation over the whole context, which is the opposite of ONTODAG’s local, incremental, mergeable discipline

— and the scaled attributes it generates would enter the canonical form, making the root depend on the fixpoint. Worth reading before designing anything here; not worth importing.

8.11 The shopping list, ranked

Table 5: Everything in Section 8, sorted by value per unit of risk. The top three cost nothing and can be done now.

Item	Verdict	Why, in one line
Pattern structures (§8.3)	Adopt (theory)	ONTODAG already built one; the theory says what is true about it and how to extend it to several coordinates.
Complexity results (§8.8)	Adopt (theory)	Turns a design instinct into a citable argument, and names the cliffs.
Scaling vocabulary (§8.2)	Adopt (core)	Reveals a real gap: there is no ordinal kind.
Reduced labelling (§8.7)	Adopt (surface)	Strictly better pictures for free.
Stability & support (§8.4)	Adopt (derived)	Half of it already exists; ranks which meets and cones deserve attention.
Implication lint (§8.1)	Adopt (derived)	Real classification help; must stay a suggestion, because implications are non-monotone.
MDL concept learning (§8.5)	Adopt (derived)	Answers “where do categories come from?”; needs a pinned root and an author to promote its output.
Attribute exploration (§8.6)	Research	The best fit with the agents story; single-writer case is buildable now.
Triadic analysis of provenance	Research	Right shape for “who agrees with whom”; must not touch the knowledge root.
Relational Concept Analysis	Research	The relations wall; global fixpoint conflicts with local canonical form.
Fuzzy FCA	Refuse	Numbers in identity destroy exact canonicalisation and merge.
Full closure in the store	Refuse	Exponential, coincidence-prone, unstable under edit, unattributable.

The pattern across the table is worth stating explicitly, because it is the answer to “how much of FCA should ONTODAG adopt?”.

Key idea

Almost all of FCA’s theory is adoptable; almost none of its representation is. The theorems, the vocabulary, the algorithms, the selection criteria and the interactive protocol can all be imported. The one thing that cannot is the concept lattice itself as a stored object — because storing it would forfeit the cheap canonical form, and the cheap canonical form is the thing everything in Part IV is built on.

9 What OntoDAG must keep

A shopping list is dangerous without a list of things that are not for sale. This section names the seven commitments that give ONTODAG its advantage, says what each one is worth, and — more usefully — says what specific temptation would destroy it. They are ordered by how expensive the mistake would be.

9.1 The cheap canonical form

The commitment. The stored form is the transitive reduction of the asserted relation, and nothing else; values are canonicalised by denotation; the serialisation is deterministic. Same knowledge, same bytes, same hash.

What it is worth. Everything downstream. Content addressing, snapshot identity, cheap diffing, structural sharing, agreement-by-hash, convergent merge, and every certificate in Part IV rests on this one property. It is also unusual: neither a relational database nor a triple store nor a graph database offers “equal meaning, equal address”.

The temptation that would destroy it. Expressiveness — and the forty-year record is worth putting beside it. The knowledge bases that actually run the world are the *weak* ones: SNOMED CT (Spackman et al., 1997), the Gene Ontology (Ashburner et al., 2000), WordNet (Miller, 1995), schema.org (Guha et al., 2016) and Wikidata (Vrandečić and Krötzsch, 2014) are, in logical terms, little more than subsumption plus vocabulary. Cyc (Lenat, 1995) bet on expressiveness plus hand-built coverage; language models then commoditised the coverage, while the expressiveness left the knowledge base hard to maintain and harder to share. Every step up the ladder — arbitrary logical axioms, rules, defaults, weights — degrades the root from a fingerprint of *knowledge* to a fingerprint of *phrasing*, because canonicalising up to logical equivalence means canonicalising an entailment closure. The two-axis test is worth memorising:

Key idea

A feature may enter the shared store only if it is **(1) monotone** — merge-as-union survives — and **(2) cheaply, semantically canonicalisable** — “equal knowledge, equal root” stays decidable and cheap.

Both are necessary. Neither is sufficient: features that pass both should still wait for a real consumer.

9.2 Names as identity

The commitment. A node *is* its name at every boundary. Names are arbitrary and stable; meaning is carried by the structure around them, not inside them.

What it is worth. Merge is possible (§7.4), references survive, and provenance has a subject. The contrast with FCA, where a concept is its extent–intent pair and therefore cannot survive an edit, is the sharpest single difference between the two systems.

The temptation. Meaning-bearing identifiers — codes that encode the classification, so that similar things get similar names. It is a very old temptation and it has a very consistent record of failure (Eco, 1995):

Attempt	How it failed
Wilkins’s philosophical language (1668) (Wilkins, 1668)	A word’s letters walk down the taxonomy — so a concept with two legitimate genera has no legal name. <i>The multi-parent problem, 1668 edition.</i>
Dewey decimal (1876)	Meaning in the identifier bought real shelf locality, and paid for it with whole fields wedged into divisions chosen in the 1870s.
Linnaean binomials (1753)	One level of meaning in the name; taxonomic revisions therefore rename species, and synonymy databases exist to track the churn.
IUPAC names vs. CAS numbers	Chemistry has the purest meaning-bearing names anywhere — and still keeps an arbitrary registry number beside them, because structures get re-perceived and databases need stable join keys.

There is also a hard mathematical reason, not merely a historical one: a prefix or path code embeds a *tree* order, and no linear key space can embed a partial order with genuine multiple parents. A child would have to share a prefix with both parents, which forces the parents to share it with each other. So meaning-bearing names cannot represent a DAG at all — which is why Wilkins agonised over placements.

What to do instead. Exactly what chemistry does: keep both. An arbitrary stable name for identity, and a derived, regenerable, meaning-bearing code (the ancestor-set code of Section 6) as an index. That settlement is available precisely because Proposition 6.8 says the code loses nothing.

9.3 Assertion over derivation

The commitment. The store holds what was said. Consequences are computed, never stored; nothing appears in the graph without an author.

What it is worth. Attributability, bounded size, stability under edit, and a coherent story about responsibility — which becomes acute the moment the writers are machines (§10.8).

The temptation. Convenience. It is always tempting to materialise a derived edge “just this once” for speed. The discipline that makes this safe already exists and should be applied without exception: derived content lives in a separate store with its own root, is never merged, and is regenerable from the pinned root it was computed against. If derived content must be shared, it enters as an ordinary claim by an identified author.

9.4 One query primitive

The commitment. Conjunctive query is intersection of cones. There is no second query mechanism.

What it is worth. It is easy to underrate this, so here is the concrete payoff. Because there is exactly one primitive:

- the planner is small enough to be *proved* result-preserving, and is tested against a brute-force oracle over every one-, two- and three-term query on fixtures and random graphs;
- typed values did not need a second mechanism — a constraint like `weight(..5kg)` is a query term, so matching is still one intersection, with one planner and one merge story;
- disjunction is a thin layer that touches nothing stored;
- consistency checking is the same primitive again: if disjointness claims are ever added, “is anything both a Cat and a Dog?” is `get(Cat, Dog)`. The check needs no checker.

The temptation. A second query path for a special case — attribute payloads with a filter step, a text index consulted separately, a graph traversal language. Each would double the surface that must agree with itself.

9.5 Mergeability

The commitment. Merge is union followed by re-reduction: commutative, idempotent, convergent, always total.

What it is worth. Multi-writer collaboration with no server, no consensus, and no coordination — and, as Section 11 argues, this is the property that makes ONTODAG suit decentralised infrastructure rather than merely tolerate it.

The temptation. A merge that refuses. It is natural to want merge to reject an inconsistent result — two writers filing the same crate under two different weights, say. But a merge that can fail is not total, and a non-total merge is not a CRDT: replicas that reject different things diverge permanently. The right answer is the one ONTODAG settled on for constraints generally: *claims merge, enforcement is local*. The inconsistency arrives, is visible, and is queryable — and a reader may refuse to act on it. Merge stays total.

Watch out

An honest caveat, recorded because it is currently a documentation gap in the system rather than a design decision: ONTODAG’s `put` refuses to file one item under two incompatible values of the same dimension, but `merge` does *not* enforce that guard — so merging two stores can produce a state `put` would have refused. Under the principle just stated this is arguably correct (a refusing merge would not be total), but it should be *stated* rather than left implicit.

9.6 Exactness

The commitment. No floating point anywhere. Values are exact rationals of an SI anchor unit; comparisons are exact arithmetic; there are no weights, probabilities or confidences in the data.

What it is worth. Determinism across replicas (G3), hence canonical roots for typed values, hence the whole dimensions story. It also has an unexpected downstream benefit: a structure with no floats and one query primitive is unusually friendly to arithmetic circuits, which is what a zero-knowledge proof would need (§11.8).

The temptation. “Just a confidence score.” A number attached to a claim seems harmless and is not: it breaks exact canonicalisation (which precision? rounded how?) and breaks merge (whose number wins?). Confidence belongs on a signed assertion record, where two authors disagreeing is representable *as two authors disagreeing* rather than as an averaged scalar that belongs to nobody.

9.7 One sort

The commitment. Everything is a node; “category” and “item” are roles, not types.

What it is worth. Category hierarchies with no extra machinery, and one traversal that crosses all levels (§7.1).

The temptation. Adding a type system — classes versus individuals, or a schema layer — for the sake of validation. The cost is that every such distinction becomes something two writers can disagree about, and something the canonical form must encode.

9.8 The meta-commitment: a place to put derived things

Underneath the seven is one architectural habit that makes them affordable, and it deserves a name because it is what allows Section 8’s shopping list to be so long.

Key idea

OntoDAG has a well-defined place for derived structure, with four rules: it lives in a *separate store with its own root*; it is *regenerable* from a pinned knowledge root; it is *never merged*; and it is *evictable* without data loss. Cone summaries, descendant counts, ancestor-set codes, implication lints, MDL-learned concepts and materialised meets are all instances.

This is what makes “adopt, but not in the core” a real answer rather than a polite refusal. Because there is somewhere else to put things, ONTODAG can be generous about what it computes and strict about what it stores.

On a storage network with rent, the pattern even acquires an economic mechanism: a derived index is kept warm exactly as long as query traffic justifies paying its postage, and is allowed to lapse otherwise (§11.4). Regenerable things can be allowed to die.

Part IV — Why this matters now

10 Why this matters for AI agents

The argument of this section is a single claim with four parts:

Key idea

An AI agent’s own knowledge is **not canonical, not addressable, not verifiable, and not attributable**. Those four properties are exactly what ONTODAG provides, and they are provided *because* of the refusals in Section 9, not in spite of them.

The corollary for design: a knowledge substrate for agents should sell *guarantees*, not *expressiveness*. Reasoning is now abundant and cheap — every agent carries a flexible reasoner in its weights. Agreement is scarce and expensive.

10.1 The four missing properties

Take them one at a time, because each names a concrete failure mode.

Not canonical. Ask a model the same question twice, phrased differently, and you may get two answers that are both reasonable and mutually inconsistent. There is no normal form for what a model knows, and therefore no way to tell whether two models — or the same model on Tuesday — know the same thing. ONTODAG: equal knowledge has equal bytes (G1), so “do we agree?” is a string comparison.

Not addressable. You cannot point at a fact in a model. There is no reference you can put in a contract, cite in a document, or hand to a counterparty that denotes “the state of knowledge you were using”. ONTODAG: a root is a 32-byte reference to the entire knowledge state, immutable and retrievable forever.

Not verifiable. A model’s output cannot be checked except by another model or a human. This is the hallucination problem in its structural form (Ji et al., 2023): the failure is not that models are often wrong but that being wrong is *indistinguishable in the output* from being right. ONTODAG: an answer comes with a certificate a third party can check with nothing but the root (§10.6).

Not attributable. When a model states something, there is no answer to “who says?” beyond “the model”. In a multi-agent system, where one agent consumes another’s output, this compounds silently. ONTODAG: every write is a signed record naming its author and the root they were looking at.

10.2 This is not an argument against retrieval

The obvious objection is that retrieval-augmented generation (Lewis et al., 2020) already grounds models in external data. It does, and the two approaches are complementary rather than competing. It is worth being precise about the division of labour, because getting it wrong wastes both.

	Vector retrieval	OntoDAG query
Input	vague, natural language	exact terms
Recall	good on paraphrase and synonymy	only what was filed
Precision	approximate top- k ; no guarantee	exact: the complete intersection, or nothing
“Is this all of them?”	unanswerable	yes, with a count
“Why this result?”	similarity score	the path of asserted edges
Reproducible?	depends on index and model version	yes, given the root
Two systems agree?	incomparable spaces	compare 32 bytes

Key idea

The embedding proposes; the graph disposes. Use vector search to go from a vague human request to candidate terms; use ONTODAG to answer exactly, completely, and citably once the terms are fixed. The failure mode to avoid is using approximate retrieval for a question whose answer must be complete — “list every refundable booking” is not a similarity query, and a top- k answer to it is wrong in a way nothing downstream can detect.

10.3 What the agent surface actually looks like

ONTODAG exposes itself to agents over the Model Context Protocol (Anthropic, 2024) as `odag-mcp`. Here is a real session against the running example (transport framing removed, JSON reformatted):

```
-> {"method": "tools/call", "params": {"name": "about", "arguments": {}}}

<- { "store": ".../store.od",
    "items": 37,
    "top": ["Booked", "Japan", "Refundable", "Travel", "dimension"],
    "dimensions": {"weight": "linear-dimension", "time": "calendar-dimension",
        "geo": "prefix-dimension", ...},
    "registry_version": "4.1",
    "surface_version": "0.1",
    "contract": "0.1",
    "capabilities": ["about", "canon", "describe", "is_below",
        "overlapping", "query"],
    "root": "6f7342307f2fc4531c75f0df2465941f37822dd5f8cac5480e...",
    "annotations": {} }
```

Five design decisions are visible in that one response, and each is a deliberate answer to one of the four missing properties.

1. Discoverability without downloading. `about` lets an agent learn what a store is *about* — its top-level categories, its declared dimensions, its size — before deciding whether to query it. Note that this record is *computed on demand and never stored*: a summary living inside the store would make the root depend on a description of itself, since the item count is part of the description and the description would be part of the hash.

2. Every answer cites its root. Not as decoration. It converts an answer from a perishable claim into a permanent one: “`get(Japan, Refundable)` at root `6f734230...` was these three documents” is true forever, checkable by anyone, and immune to the store changing afterwards. An agent that caches answers with their roots has no cache-invalidation problem, because it is not caching a guess about the present; it is recording a fact about a past state.

3. The interpretation context is pinned. `registry_version` and `contract` tell the agent which arithmetic and which guarantees were in force. A verifier running different arithmetic must refuse rather than misinterpret.

4. Answers are extensible objects, never bare lists. Every response carries a namespaced `annotations` map, empty today. New information — guarantee status, economic bonds, confidence from some other system — can arrive under its own namespace without changing any existing answer shape, and readers ignore namespaces they do not know.

5. Canonical echo. Every answer restates the terms in their *stored* form:

```
-> {"name": "query", "arguments": {"terms": ["Japan", "Refundable"]}}
<- { "terms": ["Japan", "Refundable"],
    "items": ["kyoto-inn.pdf", "outbound.pdf", "rail-pass.pdf"],
    "count": 3, "truncated": false,
    "root": "6f7342307f2fc4531c75f0df2465941f37822dd5f8cac5480e...",
    "contract": "0.1", "annotations": {} }
```

With typed values the echo earns its keep: an agent that asks about `weight(400g)` is told the store calls that `weight(2/5kg)`. It has learned the canonical spelling, so its next assertion will collide correctly with everyone else’s rather than creating a near-duplicate. *The canonical echo is how an agent discovers the store’s identity conventions without being told them.*

Note also `count` beside `items`, and `truncated`. The command-line interface truncates long output by default because a human at a terminal can see that it did; the agent surface has **no default limit**, because an agent cannot see a terminal and so cannot see a truncation. When a limit is set explicitly, `count` still reports the complete size. This is a small thing that illustrates a general principle: *an interface for machines must make its own incompleteness machine-readable.*

10.4 Closing the world by pinning, not by decree

Agents constantly need non-monotone answers: “is there anything under `Japan` that is *not* Booked?”, “how many?”, “does this store contain X at all?”. Section 7.5 explained why the living store cannot answer these: in a merging system the answer can shrink when news arrives, so two replicas would disagree.

The escape is scoping rather than prohibition:

Key idea

Monotone questions may be asked of the living store. Non-monotone questions must name a root. Indexed by a root, negation, aggregation, universal quantification and absence claims are pure, deterministic, eternally recomputable facts. “`get(A and not B)` at

root 21728cd9... = {...}” is an immutable claim anyone can verify by re-execution.

The corollary for an agent is worth stating as a rule of practice: *an agent that needs a closed world does not ask the store to close. It closes the world itself, by pinning a root.* Every tool on the surface takes an `as_of` parameter for exactly this.

10.5 Fail-closed answers

`is_below` answers *true* only when the graph plus exact arithmetic witness it. *False* means “not derivable from what is here”, never “unknown but plausible” (G4).

This is the opposite of a language model’s default behaviour, and it is what makes the operation composable with a model. An agent can use the model for the plausible and the store for the certain, and — crucially — can *tell which is which*. A system where the two are interchangeable is a system where the guarantees of the exact half are worth nothing.

10.6 Verification: from “trust the store” to “trust nobody”

Because the persistence layer is a canonically encoded hash structure, “this record is in this version” has a small proof: the exact stored bytes along the one path where the record must live, checkable by recomputing hashes. Unusually, *absence* is provable too — the encoding is canonical, so a key has exactly one possible location, and exhibiting that location empty is a proof it is not there.

ONTODAG builds subsumption certificates on top. The design is *re-execution over authenticated fragments*: the prover records the real `is_below` walk, adds the order-invariant dependency closure (so a verifier whose walk explores a different path is still covered), proves every touched record, and the verifier re-runs the *real* algorithm over a store that serves only proof-verified records. Semantics stay single-sourced; a coverage gap fails verification rather than validating a wrong answer.

Measured, not asserted

Measured round trip (Experiment 9):

root	d53e10b9ee4a06edac177b23d40a3ddab9a8f9a890a4397ac16f50548acc39ea
certificate	ontodag-is-below-certificate v1, 4 records proved, registry 4.1
claim	outbound.pdf is below Travel → True
verify_below(cert, root)	True (no store, no network)
tampered (result flipped)	CertificateError
negative answer	outbound.pdf below Hotel → False, verified False

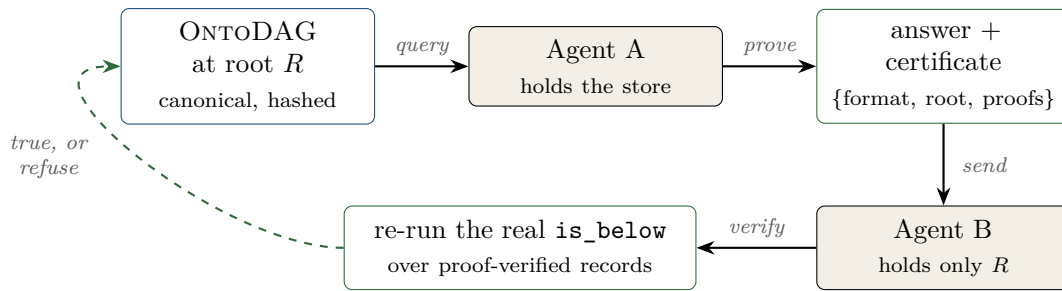
The general pattern — a data structure that ships proofs of its own answers — is what Miller et al. (2014) call an authenticated data structure; what is unusual here is that the *semantic* operation, not merely record lookup, is the thing being attested.

Both polarities work; a certificate may be incomplete (verification fails) but never misleading (a wrong answer cannot verify). So an agent can hand another agent a conclusion *and the means to check it*, without handing over the store. The answer travels; the data does not.

10.7 Writing: propose, echo, confirm

Letting agents *write* raises different problems. ONTODAG’s write surface answers three of them structurally.

Idempotence, by construction. Re-filing the same fact is a graph no-op: the same root comes out. An agent that retries after a timeout, or two agents that independently reach the



Agent B needs no copy of the store, no network access, and no trust in Agent A — only the 32 bytes of R and the verifier code. The dashed return is a claim about R, not a request to the store.

Figure 10: Verification without disclosure. Because the root commits to the whole state and the encoding is canonical, a subsumption answer can be shipped with everything needed to check it.

same conclusion, cannot corrupt anything. This falls straight out of the canonical form — it is not a feature that had to be built.

Compare-and-confirm. A write is two steps. `propose_put` returns the canonical echo (what the stored spellings will be, which claims this implies, what is already true, which parents do not exist) plus a *proposal token*: a hash of the operation, the canonicalised terms, and the current root. `put` recomputes the token; if the store has moved or a spelling resolved differently, it refuses and asks the agent to propose again.

This matters more for agents than for humans. An agent may spend many seconds of reasoning between reading and writing, during which the store can move; and it cannot notice a surprise the way a person glancing at a terminal would. Making the surprise into a refusal is the difference between a race and an error message.

Every write is a speech act. A successful write commits two things: the knowledge change, and one signed provenance record per claim, recording the author’s key, the claim (a canonical $sub \sqsubseteq sup$ pair), and the root the author was looking at when they decided. Removals emit retraction records: retracting is also something somebody did.

Notice where the signature goes. It is in a *separate* store with its own root, never inside the knowledge records — because if authorship were part of the hashed knowledge, two people who know the same thing would get different roots, and G1 would be gone. *Who said it* and *what is said* are deliberately different objects with different addresses.

10.8 The review problem

Here is the problem that will dominate agent-written knowledge, and it is not a technical one:

Key idea

Agents write faster than anyone can review. A store that accepts machine writes at machine speed accumulates claims nobody has checked, and the value of the store degrades to the value of its least careful writer.

ONTODAG offers three partial answers and one promising direction.

Attribution makes the problem tractable rather than solving it. Because every claim is signed, a reader can filter: accept claims from these keys, ignore the rest. Standing is computed from verified records only, and retraction is sticky per author. Forged records are listed but never

counted. This does not check anything; it makes *whose* unchecked claims you are carrying an explicit decision.

Endorsement is a separate speech act. A reviewer can endorse a claim without touching the knowledge graph. Reviewing is thus recorded work that accumulates, rather than an event that happens once in a chat window and is lost.

Disagreement is queryable structure. If disjointness claims are ever adopted, an inconsistency arriving by merge is not an error condition; it is a non-empty `get(Cat, Dog)`. The consistency check is the query primitive again — which means finding the mess an agent made is the same operation as using the store.

The direction: bound and order the review. Section 8.6 argued that attribute exploration is the right shape here, and the reason belongs in this section. Exploration inverts the flow: instead of the agent proposing arbitrary edits which a human must review in arbitrary order, the *system* computes the next question that is not already answered, and the agent answers it. The review queue becomes minimal (no redundant questions), ordered (each maximally informative given the answers so far), and terminating. That is the difference between review that scales and review that does not.

10.9 Agents sharing memory

The multi-agent case is where the properties compound, and it is worth a concrete scenario.

Two agents work on the same domain on different machines, offline from each other. Each files what it learns. Later they exchange roots.

1. If the roots are equal, they know the same things. *One comparison, 32 bytes, no data transferred* — and because canonicity is semantic, this works even if they built their stores in different orders, by different routes, spelling values differently.
2. If the roots differ, a structural diff localises the disagreement to the exact differing records, without downloading either store wholesale.
3. Merging converges: each folds in the other’s root and both reach the byte-identical result, whatever order the exchange happened in (G5). No server, no consensus, no lock.
4. Afterwards, each can ask “which of these claims came from the other agent, and do I trust that key?” — because the provenance store merged too, conflict-free, by the same union rule.

Compare the same scenario with two agents holding vector stores, or two fine-tuned models: there is no equality test, no diff, no merge, and no attribution. There is only “ask them both and hope”.

10.10 Token economics, briefly

A practical point that matters at scale. An exact query returns exactly the relevant items and a count. An approximate retrieval returns k chunks chosen by similarity, of which some are relevant, and the agent pays context tokens for all of them and then reasons about which to believe. For queries that are genuinely conjunctive — “refundable, in Japan, booked before March” — the exact path is both cheaper and correct, and the cost difference grows with the store.

The same applies to structure. An agent that needs to know what a store contains can read a `about` record of a few hundred tokens rather than sampling documents. Discoverability is a token-efficiency feature as much as a correctness one.

10.11 What OntoDAG does not solve

An honest list, because the value of the guarantees depends on not overstating them.

- **Proofs attest structure, never truth.** Every certificate proves what was asserted and what follows from it — never that the assertion is true of the world. That is the oracle problem, and no data structure solves it. The available answers are social (attribution and endorsement) and economic (Section 11.6).
- **Classification is not automatic.** ONTODAG has no defined concepts (§6.7), so nothing places an item in the right category for you. That job falls to the agent — which is a reasonable division of labour in 2026, but it means the quality of the store depends on the quality of the classifier.
- **Names must be agreed.** Two agents using different words for the same idea produce two nodes, and no mechanism notices. This is the price of Section 9.2, and it is real.
- **No relations.** “Alice authored Doc1” cannot be said. Agents will try; the surface logs what they fail to express, which is how the system will learn whether the wall needs moving.
- **Retraction does not travel.** A peer who still holds a removed edge re-adds it on merge. Single-writer and publish-to-readers deployments keep removals; concurrent writers do not.

11 Why this matters for cryptographic and decentralised systems

The second constituency wants the same four properties as the first, for different reasons. An agent wants canonical, addressable, verifiable, attributable knowledge because it cannot trust its own memory. A decentralised system wants it because there is no server to trust.

11.1 The gap content addressing leaves

Content-addressed storage — Merkle structures (Merkle, 1988), IPFS (Benet, 2014), Ethereum Swarm (Trón, 2020) — solved integrity and made durable, host-independent references possible. A hash names exactly one immutable byte string, forever, and tampering is detectable on retrieval.

What it did not solve is *meaning*. A 32-byte reference tells you nothing about what the content is or how it relates to anything else. The standard answer has been the manifest: a mapping from path names to references — in other words, a directory tree. And a directory tree is the wrong shape for the same reason it was always the wrong shape: every object must live in exactly one place, while almost everything worth storing is legitimately several things at once. `outbound.pdf` is a flight document, a Japan document, and a refundable booking; a tree makes you choose, or makes you duplicate.

ONTODAG is a semantic layer for exactly this gap: a small structure that records *what things are*, independently of where their bytes live, and that happens to have the one property content-addressed storage rewards above all others.

11.2 Semantic content addressing

Key idea

Ordinary content addressing gives **same bytes, same hash**. ONTODAG gives **same meaning, same hash**.

That upgrade is the whole contribution, and it is worth spelling out what it takes. Two people file the same six documents. They do it in different orders. One of them types a few redundant

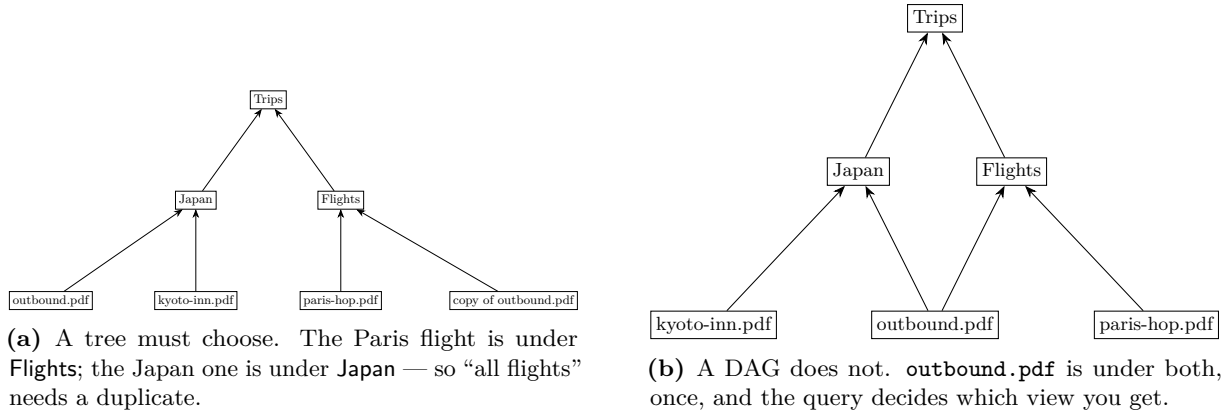


Figure 11: The manifest problem in one picture.

links along the way. One writes 400 g and the other 0.4 kg. Ordinary content addressing gives them two different hashes, because their bytes differ. ONTODAG gives them one, because:

- the transitive reduction is unique, so build order and redundant edges are quotiented away (Thm 2.11);
- values are canonicalised by denotation, so spelling is quotiented away;
- the serialisation and the storage trie are canonically encoded, so nothing about *how* the store was written survives into the bytes.

The consequences are unusual enough to list, because no mainstream database offers them:

1. **Deduplication of meaning.** Two organisations that independently build the same taxonomy converge on one address, and discover it by comparing 32 bytes.
2. **Agreement is a hash comparison.** “Do we share an ontology?” costs one string comparison and zero data transfer. Because canonicity is semantic, this is same-*meaning*-same-hash, not same-*bytes*-same-hash.
3. **A root is a commitment.** Publishing a root commits you to an entire knowledge state, in the cryptographic sense: you cannot later change what you knew without changing the reference. This is a commitment scheme you get for free, without designing one.
4. **Snapshots are free and eternal.** Nothing is overwritten, so every past root still resolves. “What did the catalogue say on the day of the contract?” is answerable years later.
5. **Disagreement localises.** Two differing roots can be narrowed to the exact differing records by structural diff, without transferring either store.

11.3 Convergence without consensus

This is the property that makes ONTODAG suit decentralised infrastructure rather than merely tolerate it, and it deserves emphasis because it is frequently assumed to be impossible.

Distributed systems usually achieve agreement by *consensus*: a protocol (or a blockchain) that decides an order of operations. Consensus is expensive, requires liveness, and requires participants to be online together.

ONTODAG needs none of it. Merge is commutative and idempotent (Thm 5.2), so replicas that have seen the same set of updates — in any order, with any duplicates — hold identical state. This is the defining property of a state-based CRDT (Shapiro et al., 2011), and combined

with content addressing it gives the pattern Sanjuán et al. (2020) call a Merkle-CRDT: state identified by hash, merged by a convergent function, gossiped rather than ordered, and which is the substrate the local-first software argument (Kleppmann et al., 2019) asks for: full local capability, collaboration without a coordinating server, and no vendor holding the authoritative copy.

Key idea

Two writers who fold in each other’s published roots reach the *byte-identical* root regardless of gossip order (G5). No server, no chain, no lock, no coordination — and afterwards they can *prove* they agree by comparing the root.

The blockchain, when one is involved at all, is therefore not needed for agreement. It is needed only for the things chains are actually good at: timestamping, settlement, and adjudicating disputes about money.

The price is stated in Section 5.6 and repeated here because it is the one thing an integrator must design around: merge is union, so it is grow-only. A removal can be resurrected by a peer holding the old state. Undoing is local. Any application that needs authoritative deletion must add its own layer.

11.4 Publishing, following, and paying rent

Concretely, on Swarm (Trón, 2020), an ONTODAG store becomes:

Swarm mechanism	What OntoDAG uses it for
chunks, retrieved by hash	the record blobs and trie nodes: one commit writes only what changed
postage stamps (storage rent)	keeping a published ontology alive; and, more interestingly, <i>not</i> keeping derived indexes alive
signed feeds	a followable address for “the latest root published by this key” — mutability on top of immutability
retrievability checks	confirming a published ontology is actually fetchable by strangers

The postage mechanism produces a genuinely elegant fit with the derived-layer discipline of Section 9. Derived structures — cone summaries, implication lints, MDL-learned concepts, materialised meets — are regenerable by definition. On a network where storage costs rent, that gives eviction a native mechanism:

Key idea

Let a derived index’s postage lapse. A cache whose eviction policy is “stop paying” is a cache with an honest cost model: an index is kept warm exactly as long as query traffic justifies its rent, and its disappearance costs nothing but recomputation. Regenerable things are allowed to die.

This also gives the workload-optimal materialisation problem of Section 12.3 a real objective function rather than a notional one: the space term is not a tuning constant, it is money.

11.5 Anchoring, and settling disputes

A root is 32 bytes, which is exactly the size of one storage word in the Ethereum Virtual Machine. Putting a root in a contract or an event log is a timestamped commitment to a whole knowledge state, at negligible cost.

What that enables is dispute resolution by proof rather than by argument:

1. A party publishes a catalogue and anchors its root on chain.

2. A counterparty acts on a claim about the catalogue — “this item is under 5 kg and located in the EU”.
3. If the claim is disputed, it is settled by exhibiting a proof against the anchored root. Inclusion, absence and subsumption all have proofs (§10.6).

Two practical tailwinds are worth recording. Swarm’s addressing is keccak-based — the EVM’s native hash — so on-chain verification of a Swarm-side inclusion proof does not require an expensive foreign hash function; and Swarm’s own storage-incentive machinery already verifies such proofs on chain, so the pieces exist.

One honest gap, though. ONTODAG’s current subsumption certificates work by *re-execution over authenticated records*, which is ideal off-chain (semantics stay single-sourced in the real implementation) and impractical on-chain (a contract cannot re-run a graph traversal). The shape that a contract *can* check is a hash-link path: a Merkle-ised index over the semantic graph, where a positive subsumption is a self-verifying chain of hashes. That is a known, designed-but-unbuilt extension, and it is the right thing to build when — and only when — somebody actually needs a subsumption verified on chain.

11.6 Bonded claims: the economic third leg

Limit L1 says proofs attest structure, never truth. Attribution (Section 10) upgrades that to “someone said it, and you may choose whose word you take”. There is a third leg available, and it is economic: *someone will pay if it is wrong*.

A sister project (**factbond**, design stage) explores bonded assertions: a stake slashed on successful dispute, a premium that prices reliability. Whatever the details, two structural gifts flow from ONTODAG’s side of the fence and are worth naming because they are unusual:

- **Claim identity is free and canonical.** Bonding a claim requires naming it unambiguously. A root plus a canonical claim is exactly that — and because a root commits to the whole store, a single bond can cover millions of facts, with any individual one disputed later by an inclusion proof. *Batched commitment, itemised dispute*.
- **One class of dispute adjudicates mechanically.** Distinguish “matches-source” claims (“the store at root *R* entails *X*”) from “matches-world” claims (“*X* is true”). The first is settled by a Tier-2 certificate — a proof checker is the cheapest possible rung of a dispute ladder, and it needs no human and no oracle. Only matches-world claims need evidence and adjudicators.

That split is only available because subsumption is decidable from the stored graph plus exact arithmetic. It is a direct dividend of the refusals in Section 9.

11.7 Marketplaces: matching as cone intersection

A second sister project (**loopmarket**) uses ONTODAG for marketplace matching, and it is the application that forced typed values into existence. A courier’s constraint — “anything up to 5 kg, collected in this region, delivered before Friday” — must match offers nobody pre-classified against that threshold.

Three properties of the result matter for a crypto audience:

- **Matching is the query primitive.** A constraint is a query term, so matching is one cone intersection — one planner, one merge story, no second filtering mechanism beside subsumption.

- **Recall-completeness is guaranteed, not hoped for.** `get_overlapping` returns every value whose denotation provably intersects the constraint’s (G6). For a marketplace, “we did not miss a match” is a property worth being able to state.
- **No floats, anywhere.** Values are exact rationals of an SI anchor unit. This is the same discipline crypto already uses for money — wei, satoshi, integer base units — adopted for the same reason: exactness by construction rather than by careful rounding.

11.8 The zero-knowledge direction

The natural next question for a privacy-sensitive counterparty is: can I prove “my catalogue contains something under weight(..5kg) and location(EU)” *without revealing the catalogue*?

This is the zero-knowledge question in its original sense (Goldwasser et al., 1989): convince a verifier that a statement holds while revealing nothing beyond its truth. It is heavy research — succinct proofs over trie walks — and ONTODAG deliberately walls it until a real consumer appears. But the positioning is worth recording, because ONTODAG is unusually well suited when the time comes:

- deterministic canonical encoding: the statement being proved is about a well-defined object, not about one serialisation among many;
- one query primitive: a single circuit shape covers the whole query language;
- **no floating point anywhere** — values are pairs of integers. Arithmetic circuits are built over finite fields, where rationals are natural and floats are a nightmare. ONTODAG’s exactness commitment (§9), adopted for canonicity, turns out to be exactly the property a proof system wants.

That last point is a good illustration of a theme in this paper: the refusals compound. Refusing floats to protect canonical roots also made the structure circuit-friendly, years before anyone asked.

11.9 Governance: canonical forms, plural

A final point, less technical but arguably the most important for a decentralised deployment.

Every attempt at a single official classification of the world has failed, and Borges gave the argument its definitive form: since we do not know what the universe is, every classification of it is arbitrary and conjectural (Borges, 1952). Foucault made the historical version of the same point — classification schemes are not refined over time so much as replaced.

The constructive reading, for a system design, is that *no single taxonomy deserves to be baked into identity*. ONTODAG’s answer is architectural rather than editorial:

- there is no official store — anyone publishes a root;
- vocabularies travel *inside* stores as ordinary declarations, so adopting a vocabulary is merging a published root, not installing software;
- adoption is by explicit reference to a root, so what you adopted is exactly what you can name and check;
- disagreement is representable: two communities may hold divergent overlays of a shared core and both be well-formed.

Wilkins wanted the one true tree. The answer here is closer to Borges: canonical *forms*, plural — one per community, per purpose, per moment — each canonical in itself, each mergeable with the others, and none final.

11.10 Honest limits, again

- **L1: proofs attest structure, never truth.** No amount of cryptography makes an assertion true. Attribution and bonds change the incentives, not the epistemology.
- **L2: everything is relative to a pinned interpretation context.** A proof about a typed value must pin the declarations and the registry version. A verifier running different arithmetic must refuse rather than misinterpret — and ONTODAG’s certificates do refuse.
- **Grow-only merge** remains the standing constraint on any multi-writer deployment.
- **The infrastructure wall is real.** Running a node, funding it, and buying storage rent is a genuine barrier — measured in this project’s own logs at minutes-to-hours for a first-time user, not seconds. The practical response has been to make the interesting properties available *before* the network step: a local content-addressed store gives canonical roots, snapshots, certificates and merge with no node at all, so the ideas can be evaluated before the infrastructure is confronted.

Part V — Directions

12 Directions

This section maps what the comparison opens up. Items are grouped by how much has to be invented before they can be built, and each carries the same four labels used in Section 8: what it is, what it buys, what it costs, and which invariant it puts at risk.

A caution first, which is this project’s own standing discipline and is worth inheriting: *nothing here is scheduled by virtue of being written down*. Each item is recorded with the trigger that would signal it has become necessary, so that a future reader recognises the moment instead of pre-building for it.

12.1 The organising picture

Almost everything below is a point on one line, so it is worth drawing the line first.

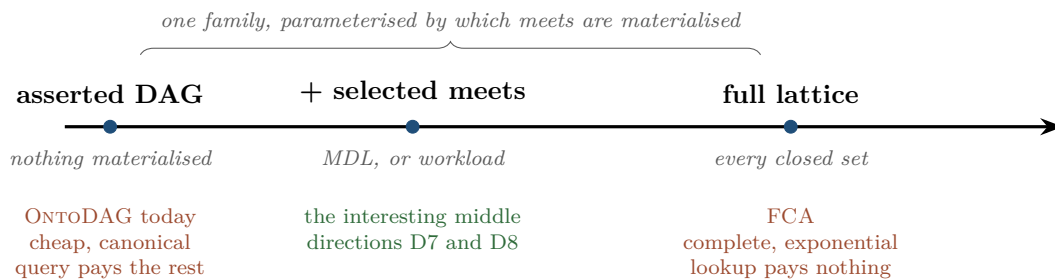


Figure 12: ONTODAG and FCA are the two endpoints of a single family, not two rival data structures. A member of the family is: the asserted graph, plus a set of materialised meets, plus a storage mode per cone. ONTODAG materialises none and pays at query time; FCA materialises all and pays in space. The research question is which middle point a given workload wants.

Seeing it this way reframes several apparently separate questions as one. “Which concepts should MDL learn?”, “which cones should be indexed?”, “which meets should the query planner materialise?” and “which categories should a user be prompted to create?” are the same question asked by four different subsystems.

12.2 Buildable now

These need no new theory and threaten no invariant.

D1. Re-describe dimensions as an interval pattern structure. *Cost: an afternoon of writing.* Section 8.3 showed ONTODAG’s typed values are an instance of Ganter and Kuznetsov (2001). Recording that in the design documents converts an engineering hunch into a known-correct construction, gives the multi-coordinate question a ready answer (product semilattice, componentwise meet), and supplies the projection theory that the future per-dimension index will need. Highest value-to-cost ratio in the paper.

D2. An ordinal kind. *Trigger: someone files ranks.* The scale table of Section 8.2 has a hole where `beginner < intermediate < expert` should be. Declaring an ordinal chain is monotone and canonicalisable, so it passes both admissibility axes. Build it with the arithmetic deliberately *absent*, because the standing hazard with ordinal data is people averaging ranks (Stevens, 1946).

D3. Reduced labelling in the visualiser. *Cost: a rendering change.* Compare Figures 5 and 7: the same information, far less ink. Worth most on stores with many typed values.

D4. An implication lint. *Trigger: users mis-filing things that the store’s own contents would have caught.* Compute extensional implications over a pinned root into a derived store; surface them as prompts. Never store them as claims (§8.1). The natural first version is single-premise implications only, which are cheap — one closure per category.

D5. Cone bitmaps behind get. *Trigger: queries measurably hot.* An in-memory bitmap per category, intersected with a bitwise AND. Bounded, dependency-free (Python integers suffice), invisible in the schema, and it has a free test oracle: the population count of a cone bitmap must equal the maintained `descendant_count`. This is the first step of the semantic-codes programme and the one that needs no decisions. When it outgrows plain integers the prior art is well mapped: compressed bitmaps (Lemire et al., 2018) for the representation, and interval labelling (Agrawal et al., 1989) for the trick of numbering nodes so that a cone becomes a few contiguous ranges.

12.3 Medium term: needs a design, not a discovery

D6. Attribute exploration with a language model as the expert. *Trigger: available now; the blocker is willingness, not knowledge.*

This is the most promising item in the paper, and Sections 8.6 and 10.8 argued why: it is the only proposal here that makes review of machine-written knowledge *bounded, ordered and terminating*. A concrete v1 for the single-writer case:

1. Pin a root. Everything that follows is relative to it, so the session is reproducible and citable.
2. View the store as the context $\mathbb{K}(P)$ of Section 6 and compute the next unconfirmed implication by `NextClosure` (Ganter, 2010).
3. Ask the agent, over the MCP surface, in domain language: *is every X also a Y ?*
4. **Yes** $\rightarrow$ record a signed confirmation in the provenance store. If the implication has a single-category premise it is also a subsumption, so offer it as an ordinary `put`.
5. **No** $\rightarrow$ require a counterexample, and file it with `put`. The store grows by exactly the object that refutes the question.
6. Terminate when no unconfirmed implications remain. Present the human with the confirmed base — which is minimal and non-redundant by construction.

Two properties make this more than a demo. The counterexample requirement means a *No* is as informative as a *Yes* — the store improves either way, which is not true of ordinary agent review. And because every answer is a signed claim against a pinned root, a later reader can see precisely which questions this author answered, and re-run the exploration under a different author to compare.

Open sub-problem: a wrong expert. Classical exploration assumes infallibility; attribution degrades the guarantee honestly to “complete relative to what this author asserted”, which is the right weakening. A second agent re-answering the same question sequence, with disagreements surfaced, is the obvious cheap defence and would be worth measuring.

D7. Workload-optimal materialisation. *Trigger: a real query log.* Figure 12’s middle, stated as an optimisation problem: choose a set M of materialised meets and a storage mode per cone to minimise

$$\mathbb{E}_{q \sim Q}[\text{retrieval cost}(q; M)] + \lambda_{\text{space}} \cdot \text{storage}(M) + \lambda_{\text{maint}} \cdot \text{update cost}(M).$$

This is a known problem three times over: view-materialisation selection on a lattice (Harinarayan et al., 1996), whose greedy solution is near-optimal by submodularity; retrieval-aware MDL, which is the same objective seen from the learner’s side; and adaptive indexing (Idreos et al., 2007), which supplies the online algorithm — every query that computes a residual intersection has just computed a candidate meet, so admit it when expected reuse covers its cost and evict when cold.

Two ONTODAG-specific notes. The selectivity statistics the greedy algorithm needs already exist and are exact (`descendant_count`). And the space term is not a tuning constant on a rent-charging network: it is postage (§11.4).

The soundness condition, which is easy to miss: materialised meets may be used to *prune* candidates but not to *generate* them, unless placement is made canonical — because of the meet-substitution trap (§6.7). A node under A and B is not the meet of A and B .

D8. MDL-learned concepts, with a promotion path. *Trigger: a store big enough that nobody can see its structure.* Run the learner (Földiák, 2026a) over a pinned root; keep its output in a derived store; let a human or an accountable agent promote a learned category into the shared graph as an ordinary claim recording the root it was derived from. The layering is what keeps this safe (§8.5), and the promotion step is what keeps it attributable.

The interesting scientific question underneath is whether MDL-selected concepts agree with the ones people actually name. That is a measurable question — take a hand-built ontology, hide the intermediate categories, learn from the leaf assignments, compare — and nobody appears to have run it at scale.

D9. Merkle-ised cone commitments. *Trigger: somebody needs a subsumption verified on chain.* A per-node cone-commitment index, in its own store, manifest-pinned, deterministic from the canonical form. It buys path-sized order-free positive proofs, per-category agreement by one hash comparison, and the one proof shape a contract can check (§11). It can never be the base representation — names are identity, and a Merkle address changes whenever anything below the node changes — so it is an index over the named graph, forever.

D10. The thin browser client. *Trigger: a published store anyone should be able to query without downloading.* Most pieces exist: a lazy reader that fetches records as a query walks them, published cone summaries that collapse broad queries to a handful of fetches, and a base install that is pure Python. The remaining work is batching the fetch frontier and bridging to network access from inside a browser sandbox.

12.4 Longer term: genuinely open

D11. Concurrent exploration. Two agents exploring two replicas ask different questions and confirm different things. Since exploration assumes a fixed universe and ONTODAG assumes a merging one, reconciling them is open. A promising angle: exploration confirmations are claims, claims merge conflict-free, and the *question* sequence is a derived, local artefact — so perhaps only the answers need to converge, and each replica may legitimately have asked a different route to them.

D12. Relations. The biggest wall. “Alice authored Doc1” is not subsumption, and no amount of cleverness makes it one. Two things are known. The FCA-native answer, Relational Concept Analysis (Rouane-Hacene et al., 2013), computes a global fixpoint, which conflicts with local canonical form. And the description-logic answer (Baader et al., 2003), an EL-shaped extension, is *monotone* — so merge would survive — but makes subsumption a global inference, so canonicalising up to logical equivalence means canonicalising an entailment closure. That is the axis that breaks, and naming it precisely is the contribution of Section 7.3: the guarantee would silently degrade from “equal knowledge, equal root” to “equal syntax, equal root”, which is the weaker thing consumers would mistakenly keep relying on. Anyone attempting this should treat cheap semantic canonicalisation of the fragment as the *primary* research problem, not tractability of reasoning.

There is a precise precedent for drawing a line this way: EL++ retains polynomial subsumption only under “p-admissible” concrete domains (Baader et al., 2005) — the description-logic community’s version of ONTODAG’s rule that only exact-arithmetic kinds may enter the canonical order.

D13. Zero-knowledge subsumption. *Trigger: a privacy-demanding counterparty.* Positioned in Section 11.8; the encouraging part is that the properties a circuit wants — determinism, one primitive, no floats — were already bought for other reasons.

D14. Products of DAGs, and factorial codes. This one returns to the origin. Section 3 described a code whose active units carry a category structure; the natural next question from the coding side is whether that structure *factors* — whether the code decomposes into independent groups of units, each carrying its own small DAG, so that an item’s description is a tuple rather than a set. That is a factorial code in Barlow’s sense (Barlow, 1989), a product lattice in order-theoretic terms, and a product of pattern structures in FCA terms.

ONTODAG already has a degenerate version: its dimensions are exactly such independent factors (weight, time and location vary independently, and a value in one says nothing about the others). Generalising from a handful of declared dimensions to *learned* factors would connect the MDL programme to the redundancy-reduction tradition it came from, and would give the surface layer a principled account of when two categories are “about the same thing”. The machine-learning neighbours to read first are the layer-wise total-correlation approach of Ver Steeg and Galstyan (2014) and the unbounded-depth Bayesian prior over binary latent DAGs of Adams et al. (2010).

D15. Continuous relaxations as a recall layer. Order embeddings (Vendrov et al., 2016) and hyperbolic embeddings (Nickel and Kiela, 2017) learn vectors in which subsumption is coordinate dominance or radial position. They are the approximate, differentiable cousins of the ancestor-set code, and they fail in exactly the way the exact structure does not: approximately. That makes them a natural *proposal* layer over an exact *disposal* layer, which is precisely the division of labour Section 10 recommends for retrieval. Worth noting that the tree/DAG obstruction reappears there too — trees embed in hyperbolic space with arbitrarily low distortion, DAGs only approximately.

D16. Closing the mind–brain loop. The most speculative and, for this project’s origins, the most fitting. Figure 4 has two directions and the literature has mostly travelled one of them: from recorded codes to concept lattices (Endres and Földiák, 2008; Endres et al., 2010). The other direction is now buildable: take an ontology, embed it as an ancestor-set code (Proposition 6.8 says nothing is lost), and ask whether a population code with that structure predicts responses better than one without. ONTODAG would be supplying the ontology in a form where the embedding is exact and reproducible — which is to say, canonical — and where two laboratories could confirm they used the identical ontology by comparing 32 bytes.

12.5 The map

Table 6: All directions, with the trigger that would justify starting.

	Direction	Risk to in-variants	Trigger
D1	Pattern-structure framing	none	now (free)
D2	Ordinal kind	none	someone files ranks
D3	Reduced labelling	none	now
D4	Implication lint	none (derived)	users mis-filing
D5	Cone bitmaps	none (derived)	queries hot
D6	Exploration with an LLM expert	none (claims)	now; the best fit with agents
D7	Workload materialisation	soundness caveat	a real query log
D8	MDL concepts + promotion	none (derived)	store too big to see
D9	Merkle cone commitments	none (derived)	on-chain subsumption needed
D10	Thin browser client	none	a published store worth reading
D11	Concurrent exploration	unclear	multi-writer exploration
D12	Relations	breaks canonicity	mass reification observed
D13	Zero-knowledge subsumption	none	a private counterparty
D14	Products of DAGs	none (derived)	research
D15	Embeddings as recall	none (outside)	vague-input queries
D16	Ontology $\rightarrow$ neural code	none	a collaborator with data

Only one row in that table says “breaks canonicity”, and it is the one everybody asks for first. That is the paper’s practical warning, and Section 13 makes it the closing point.

13 Conclusion

The question this paper set out to answer was how ONTODAG relates to Formal Concept Analysis. The answer turned out to be more specific than expected, and more useful.

They compute the same function. FCA’s derivation operator on attribute sets *is* ONTODAG’s query primitive: $B' = \text{get}(B)$, exactly (Proposition 6.3). An FCA intent *is* an ancestor-set code; an extent *is* a descendant cone; and the concept lattice of an ONTODAG is its Dedekind–MacNeille completion (Theorem 6.5). The reflexive ancestor sets are a lossless re-encoding of the graph, verified on 200 random cases. So the intuition that started the project — that a sparse binary code carries a category structure in its overlaps — is exactly right, and FCA is its mathematics.

They differ in when the function is computed. FCA evaluates it for every closed set, in advance, and stores the results. ONTODAG evaluates it per query and stores nothing. Everything else follows: the size difference (linear against up to exponential, measured in Section 4.9), the canonical-form difference (a unique transitive reduction you can hash against a complete lattice you cannot), the identity difference (stable names against extent–intent pairs that do not survive an edit), and the mergeability difference (a CRDT against a structure with no merge at all).

Almost all of FCA’s theory is adoptable; almost none of its representation is. Section 8 found seven imports worth making — pattern structures as the correct account of typed values, the scaling vocabulary and the ordinal kind it reveals as missing, stability and support as selection criteria, implication bases as a local lint, reduced labelling for diagrams, the complexity results as guardrails, and attribute exploration as a protocol — and every one of them lands in a derived, local, regenerable layer, or in the documentation, or in the interface. None of them lands in the stored graph. That is not a coincidence; it is the discipline that made the list safe to write.

The refusals are the product. ONTODAG’s value to an AI agent is that its knowledge is canonical, addressable, verifiable and attributable — four properties a language model’s knowledge lacks, and four properties that exist only because the system declines to store derived content, declines to put numbers in identity, declines to let merge fail, and declines to grow past the largest fragment where “equal knowledge, equal root” stays cheap and semantic. Its value to a decentralised system is the same list read differently: same meaning gives same hash, so agreement is a comparison and not a negotiation; merge converges, so consensus is needed only for money; and there are no floats anywhere, so the structure is unusually ready for the proof systems nobody has asked for yet.

If there is one sentence to carry away, it is the trade in Section 7.4, because it explains why two formalisms this closely related ended up serving different masters:

Key idea

FCA derives concepts and therefore cannot name them stably. ONTODAG names concepts and therefore cannot derive them.

FCA is a discovery instrument. ONTODAG is an agreement instrument.

The interesting engineering is in connecting them — letting FCA discover, in a derived layer over a pinned root, and letting an accountable author promote what it discovers into a graph that anyone can agree with, cite, verify and merge. Neither half does that alone. Section 12 is a list of ways to build the joint, and the one worth starting is the oldest idea in either field: let the machine ask the questions, in minimal non-redundant order, and let a named author answer them.

References

- Ryan P. Adams, Hanna M. Wallach, and Zoubin Ghahramani. Learning the structure of deep sparse graphical models. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1–8, 2010.
- Rakesh Agrawal, Alexander Borgida, and H. V. Jagadish. Efficient management of transitive relationships in large data and knowledge bases. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 253–262, 1989.
- Alfred V. Aho, Michael R. Garey, and Jeffrey D. Ullman. The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137, 1972.
- Hassan Aït-Kaci, Robert Boyer, Patrick Lincoln, and Roger Nasr. Efficient implementation of lattice operations. *ACM Transactions on Programming Languages and Systems*, 11(1):115–146, 1989.
- Anthropic. Model context protocol, 2024. Specification, <https://modelcontextprotocol.io>.
- Michael Ashburner, Catherine A. Ball, Judith A. Blake, et al. Gene ontology: Tool for the unification of biology. *Nature Genetics*, 25(1):25–29, 2000.
- Franz Baader, Diego Calvanese, Deborah L. McGuinness, Daniele Nardi, and Peter F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation and Applications*. Cambridge University Press, 2003.
- Franz Baader, Sebastian Brandt, and Carsten Lutz. Pushing the EL envelope. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 364–369, 2005.
- H. B. Barlow. Single units and sensation: A neuron doctrine for perceptual psychology? *Perception*, 1(4):371–394, 1972.
- H. B. Barlow. Unsupervised learning. *Neural Computation*, 1(3):295–311, 1989.
- Juan Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.
- Garrett Birkhoff. *Lattice Theory*, volume 25 of *American Mathematical Society Colloquium Publications*. American Mathematical Society, Providence, RI, 1940.
- Jorge Luis Borges. El idioma analítico de John Wilkins. In *Otras inquisiciones*. Sur, Buenos Aires, 1952. Essay first published 1942.
- Radim Bělohlávek. Concept lattices and order in fuzzy logic. *Annals of Pure and Applied Logic*, 128(1–3):277–298, 2004.
- Radim Bělohlávek and Vilém Vychodil. Discovery of optimal factors in binary data via a novel method of matrix decomposition. *Journal of Computer and System Sciences*, 76(1):3–20, 2010.
- Claudio Carpineto and Giovanni Romano. *Concept Data Analysis: Theory and Applications*. Wiley, 2004.
- E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.

- Umberto Eco. *The Search for the Perfect Language*. Blackwell, Oxford, 1995.
- Dominik Endres and Peter Földiák. Interpreting the neural code with formal concept analysis. In *Advances in Neural Information Processing Systems 21 (NIPS 2008)*, pages 425–432, 2008.
- Dominik Endres, Peter Földiák, and Uta Priss. An application of formal concept analysis to semantic neural decoding. *Annals of Mathematics and Artificial Intelligence*, 57(3–4):233–248, 2010.
- Péter Földiák. Forming sparse representations by local anti-Hebbian learning. *Biological Cybernetics*, 64(2):165–170, 1990.
- Péter Földiák. Sparse neural representation for semantic indexing, 2003. XIII Conference of the European Society for Cognitive Psychology (ESCOP), Granada.
- Péter Földiák. mdl-fca: Minimum-description-length concept learning over binary data, 2026a. <https://github.com/petfold/mdl-fca>.
- Péter Földiák. OntoDAG: A DAG-based associative memory and category manager, 2026b. <https://github.com/petfold/ontodag>.
- Péter Földiák. OntoDAG: A semantic layer for distributed storage, 2026c. Companion manuscript; see `paper/main.tex` in <https://github.com/petfold/ontodag>.
- Péter Földiák. recordstore: A versioned record store over content-addressed storage, 2026d. <https://github.com/petfold/recordstore>.
- Bernhard Ganter. Two basic algorithms in concept analysis. In *Formal Concept Analysis (ICFCA 2010)*, volume 5986 of *LNCS*, pages 312–340. Springer, 2010. Originally FB4-Preprint No. 831, TH Darmstadt, 1984.
- Bernhard Ganter and Sergei O. Kuznetsov. Pattern structures and their projections. In *Conceptual Structures: Broadening the Base (ICCS 2001)*, volume 2120 of *LNCS*, pages 129–142. Springer, 2001.
- Bernhard Ganter and Sergei O. Obiedkov. *Conceptual Exploration*. Springer, Berlin, 2016.
- Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, Berlin, 1999. Translated by C. Franzke.
- Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, Cambridge, MA, 2007.
- R. V. Guha, Dan Brickley, and Steve Macbeth. Schema.org: Evolution of structured data on the web. *Communications of the ACM*, 59(2):44–51, 2016.
- Jean-Louis Guigues and Vincent Duquenne. Familles minimales d’implications informatives résultant d’un tableau de données binaires. *Mathématiques et Sciences Humaines*, 95:5–18, 1986.
- Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. Implementing data cubes efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pages 205–216, 1996.

- Stratos Idreos, Martin L. Kersten, and Stefan Manegold. Database cracking. In *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 68–78, 2007.
- Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248:1–248:38, 2023.
- Pentti Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2):139–159, 2009.
- Mehdi Kaytoue, Sergei O. Kuznetsov, Amedeo Napoli, and Sébastien Duplessis. Mining gene expression data with pattern structures in formal concept analysis. *Information Sciences*, 181(10):1989–2001, 2011.
- Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178, 2019.
- Petr Krajca, Jan Outrata, and Vilém Vychodil. Advances in algorithms based on CbO. In *Concept Lattices and Their Applications (CLA 2010)*, pages 325–337, 2010.
- Sergei O. Kuznetsov. On computing the size of a lattice and related decision problems. *Order*, 18(4):313–321, 2001.
- Sergei O. Kuznetsov. On stability of a formal concept. *Annals of Mathematics and Artificial Intelligence*, 49(1–4):101–115, 2007.
- Sergei O. Kuznetsov and Sergei A. Obiedkov. Comparing performance of algorithms for generating concept lattices. *Journal of Experimental & Theoretical Artificial Intelligence*, 14(2–3):189–216, 2002.
- Fritz Lehmann and Rudolf Wille. A triadic approach to formal concept analysis. In *Conceptual Structures: Applications, Implementation and Theory (ICCS 1995)*, volume 954 of *LNCS*, pages 32–43. Springer, 1995.
- Gottfried Wilhelm Leibniz. *Dissertatio de arte combinatoria*. J. S. Fick and J. P. Seubold, Leipzig, 1666.
- Daniel Lemire, Owen Kaser, Nathan Kurz, Luca Deri, Chris O’Hara, François Saint-Jacques, and Gregory Ssi-Yan-Kai. Roaring bitmaps: Implementation of an optimized software library. *Software: Practice and Experience*, 48(4):867–895, 2018.
- Douglas B. Lenat. CYC: A large-scale investment in knowledge infrastructure. *Communications of the ACM*, 38(11):33–38, 1995.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, 2020.
- Christian Lindig. Fast concept analysis. In *Working with Conceptual Structures — Contributions to ICCS 2000*, pages 152–161. Shaker Verlag, 2000.
- H. M. MacNeille. Partially ordered sets. *Transactions of the American Mathematical Society*, 42(3):416–460, 1937.
- Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO ’87*, volume 293 of *LNCS*, pages 369–378. Springer, 1988.

- Pauli Miettinen and Jilles Vreeken. MDL4BMF: Minimum description length for Boolean matrix factorization. *ACM Transactions on Knowledge Discovery from Data*, 8(4):18:1–18:31, 2014.
- Andrew Miller, Michael Hicks, Jonathan Katz, and Elaine Shi. Authenticated data structures, generically. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 411–423, 2014.
- George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41, 1995.
- Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems 30*, pages 6338–6347, 2017.
- Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381:607–609, 1996.
- Nicolas Pasquier, Yves Bastide, Rafik Taouil, and Lotfi Lakhal. Efficient mining of association rules using closed itemset lattices. *Information Systems*, 24(1):25–46, 1999.
- Tony A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.
- Jonas Poelmans, Dmitry I. Ignatov, Sergei O. Kuznetsov, and Guido Dedene. Formal concept analysis in knowledge processing: A survey on applications. *Expert Systems with Applications*, 40(16):6538–6560, 2013.
- Uta Priss. Formal concept analysis in information science. *Annual Review of Information Science and Technology*, 40(1):521–543, 2006.
- S. R. Ranganathan. *Colon Classification*. Madras Library Association, Madras, 1933.
- Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.
- Mohamed Rouane-Hacene, Marianne Huchard, Amedeo Napoli, and Petko Valtchev. Relational concept analysis: Mining concept lattices from multi-relational data. *Annals of Mathematics and Artificial Intelligence*, 67(1):81–108, 2013.
- Héctor Sanjuán, Samuli Pöyhtäri, Pedro Teixeira, and Ioannis Psaras. Merkle-CRDTs: Merkle-DAGs meet CRDTs, 2020. arXiv:2004.00107.
- Marc Shapiro, Nuno Pregoça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *LNCS*, pages 386–400. Springer, 2011.
- Koen Smets and Jilles Vreeken. Slim: Directly mining descriptive patterns. In *Proceedings of the 2012 SIAM International Conference on Data Mining (SDM)*, pages 236–247, 2012.
- Gregor Snelting and Frank Tip. Understanding class hierarchies using concept analysis. *ACM Transactions on Programming Languages and Systems*, 22(3):540–582, 2000.
- Kent A. Spackman, Keith E. Campbell, and Roger A. Côté. SNOMED RT: A reference terminology for health care. In *Proceedings of the AMIA Annual Fall Symposium*, pages 640–644, 1997.
- S. S. Stevens. On the theory of scales of measurement. *Science*, 103(2684):677–680, 1946.
- Gerd Stumme, Rafik Taouil, Yves Bastide, Nicolas Pasquier, and Lotfi Lakhal. Computing iceberg concept lattices with TITANIC. *Data & Knowledge Engineering*, 42(2):189–222, 2002.

- Viktor Trón. *The Book of Swarm: Storage and Communication Infrastructure for a Self-Sovereign Digital Society*. Swarm Foundation, 2020. <https://www.ethswarm.org/the-book-of-swarm.pdf>.
- Ivan Vendrov, Ryan Kiros, Sanja Fidler, and Raquel Urtasun. Order-embeddings of images and language. In *International Conference on Learning Representations (ICLR)*, 2016.
- Greg Ver Steeg and Aram Galstyan. Discovering structure in high-dimensional data through correlation explanation. In *Advances in Neural Information Processing Systems 27*, 2014.
- Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
- Jilles Vreeken, Matthijs van Leeuwen, and Arno Siebes. Krimp: Mining itemsets that compress. *Data Mining and Knowledge Discovery*, 23(1):169–214, 2011.
- W3C OWL Working Group. OWL 2 web ontology language document overview (second edition), 2012. W3C Recommendation, <https://www.w3.org/TR/owl2-overview/>.
- John Wilkins. *An Essay Towards a Real Character, and a Philosophical Language*. The Royal Society, London, 1668.
- Rudolf Wille. Restructuring lattice theory: An approach based on hierarchies of concepts. In Ivan Rival, editor, *Ordered Sets*, pages 445–470. Reidel, Dordrecht, 1982.

A Glossary

Terms are grouped by where they come from. A term defined in the body carries its section number.

Order theory

Antisymmetry

If $x \leq y$ and $y \leq x$ then $x = y$. In a stored DAG this is exactly acyclicity (§2).

Complete lattice

A poset in which *every* subset — not merely every pair — has a meet and a join. Automatically has a top and a bottom.

Cone

For a node x : the descendant cone $\downarrow x$ is everything reachable from x (including x); the ancestor cone $\uparrow x$ is everything from which x is reachable. ONTODAG’s query answers are intersections of descendant cones (§5.3).

Cover

y covers x when $x < y$ with nothing strictly between. The covering pairs are exactly the edges of a Hasse diagram, and exactly the edges of a transitive reduction.

DAG

Directed acyclic graph. Reachability in a DAG is a partial order, and every finite poset arises this way (Prop. 2.7).

Dedekind–MacNeille completion

The smallest complete lattice into which a poset embeds, preserving all meets and joins it already had. Equal to the concept lattice of $(P, P, \leq)$ (Thm 6.5); MacNeille (1937).

Galois connection

A pair of containment-reversing maps between the subsets of two sets, such that $A \subseteq g(B) \iff B \subseteq f(A)$. Both composites are closure operators (§2.7).

Hasse diagram

A drawing of a poset showing only covering pairs, with greater elements higher. All other relationships are read by walking.

Lattice

A poset in which every pair has a meet and a join.

Meet / join

Greatest lower bound / least upper bound of a set of elements, when one exists.

Partial order

Reflexive, antisymmetric, transitive relation. “Partial” because two elements may be incomparable — which is the point.

Poset

A set with a partial order.

Transitive closure

Add an edge wherever a path exists.

Transitive reduction

Remove every edge implied by a path. For a DAG it is unique and equals the Hasse diagram (Thm 2.11; Aho et al., 1972).

Formal Concept Analysis**Attribute exploration**

An interactive protocol in which the system asks an expert whether each unconfirmed implication always holds, and the expert either confirms it or supplies a counterexample (§8.6; Ganter and Obiedkov, 2016).

Concept lattice $\mathfrak{B}(\mathbb{K})$

All formal concepts of a context, ordered by extent containment. Always a complete lattice (Thm 4.7).

Contranominal scale

The context $(\{1..n\}, \{1..n\}, \neq)$. Every attribute set is closed, so it has 2^n concepts — the canonical worst case (§4.8).

Derivation ($'$)

A' is the attributes shared by all of A ; B' is the objects having all of B . In ONTODAG terms $B' = \text{get}(B)$ (Prop. 6.3).

Duquenne–Guigues base (stem base)

The unique minimum-size set of implications generating all implications of a context (Guigues and Duquenne, 1986).

Extent / intent

The two halves of a formal concept: the objects it covers, and the attributes defining it.

Formal concept

A pair (A, B) with $A' = B$ and $B' = A$ — a query in balance with its own answer (§6).

Formal context (G, M, I)

Objects, attributes, and which has which. The same object as a binary code matrix (§3.3).

Iceberg lattice

The concepts above a support threshold (Stumme et al., 2002).

Implication $B \rightarrow C$

Every object with all of B has all of C . Non-monotone: new data can destroy it (§8.1).

NextClosure

Ganter’s algorithm enumerating all closed sets in lexic order, in constant memory (Ganter, 2010).

Pattern structure

FCA generalised so descriptions live in a meet-semilattice rather than being attribute sets; the interval case matches ONTODAG’s typed values (§8.3; Ganter and Kuznetsov, 2001; Kaytoue et al., 2011).

Reduced labelling

Drawing convention where each object and attribute name appears once, at the concept that introduces it (§4.4).

Scaling

Turning non-binary data into a context by choosing, per attribute, which comparisons matter (§8.2).

Stability

The fraction of subsets of a concept’s extent that still close to the same intent: robustness against losing objects (Kuznetsov, 2007).

OntoDAG**Ancestor-set code**

$\text{code}(x) = \uparrow x$, the reflexive ancestor set read as a bit vector. Equal to the FCA intent; determines the graph (Prop. 6.8).

Canonical form

The unique transitive reduction plus denotation-canonicalised values plus a deterministic serialisation. What makes a single root hash possible (§5.4).

Certificate

A self-describing envelope carrying the raw record bytes needed to re-check an `is_below` answer against a root alone (§10.6).

Cone summary

A published derived record stating a broad category’s membership, so a thin client need not walk it.

descendant_count

The exact size of a node’s cone, delta-maintained. FCA’s support, and the query planner’s statistics.

Dimension / parametric value

A typed value such as `weight(4kg)` whose order is computed from the name by exact rational arithmetic rather than stored as edges (§5.7).

get_overlapping

Candidate generation for parametric terms: complete for possibility, silent on satisfaction (G6).

is_below

Boolean subsumption, answered by an upward walk, fail-closed (G4).

Merge

Union of assertions followed by re-reduction. Commutative and idempotent, hence a CRDT (Thm 5.2).

Pinned root / as-of

Asking a question against an immutable past state, which is how non-monotone questions become well defined (§10.4).

Provenance record

A signed statement that an author asserted (or retracted, or endorsed) a claim, against the root they were looking at. Stored separately from the knowledge (§10).

Residency

Whether a graph is eager (fully in memory), lazy (fetched as a query walks), or sparse (partially resident and writable). Semantics are identical across all three (G3).

Root

The 32-byte content address of an entire knowledge state.

*

The graph root node, implicit ancestor of everything.

Distributed systems and cryptography**Content addressing**

Retrieving a blob by the hash of its own bytes; nothing is ever overwritten.

CRDT

Conflict-free replicated data type: replicas converge under a commutative, idempotent merge, with no coordination (Shapiro et al., 2011).

Feed

A signed, followable pointer to a changing value — here, “the latest root published by this key”.

Merkle proof

The path of hashes showing a record is (or, given a canonical encoding, is not) part of a hashed structure (Merkle, 1988).

Monotone

An answer that can only grow as information arrives. Required of anything asked of a merging store (G2).

Postage stamp

Prepaid storage rent on Swarm; the natural eviction mechanism for regenerable derived indexes (§11.4).

B Notation

A phrasebook for a reader who knows one field and is reading about the other. The middle column is what this paper writes; the outer columns are what each community usually says.

Table 7: One object, three vocabularies.

FCA says	This paper writes	OntoDAG says
formal context $\mathbb{K} = (G, M, I)$	$\mathbb{K}(P) = (P, P, \sqsubseteq)$	the graph
object $g \in G$	node x	an item
attribute $m \in M$	node y	a category
$g I m$	$x \sqsubseteq y$	x is under y
extent A	$\downarrow y$	the cone of y ; what <code>get(y)</code> returns
intent B	$\text{code}(x) = \uparrow x$	the ancestor set; the code
B'	$\bigcap_{y \in B} \downarrow y$	<code>get(B)</code>
A'	$\bigcap_{x \in A} \text{code}(x)$	the categories all of A share
closure B''	—	the closed form of a query
$(A_1, B_1) \leq (A_2, B_2)$	$x \sqsubseteq y$	x is below y
concept lattice $\mathfrak{B}(\mathbb{K})$	$\mathfrak{B}(\mathbb{K}(P))$	the completion (not stored)
Hasse diagram of $\mathfrak{B}$	G^{red}	the stored edges
support $ A $	$ \downarrow y $	<code>descendant_count</code>
implication $B \rightarrow C$	$C \subseteq B''$	(no stored counterpart)

Symbols used in the text

$\subseteq, \subsetneq$	containment, strict	the shape of every “is-a”
$\cap, \cup$	intersection, union	query, merge
$\leq, <, \leq$	order, strict order, cover	§2
$\sqsubseteq$	subsumption between nodes	“ x is a special case of y ”
$\downarrow y$	descendant cone of y	the query answer
$\uparrow x$	ancestor cone of x	the code
$\text{code}(x)$	$\uparrow x$ read as a bit vector	§6
$\wedge, \vee$	meet, join	greatest lower / least upper bound
$\top, \perp$	top, bottom	of a complete lattice
A', B', B''	derivation, closure	§4.2
$\mathbb{K}, \mathfrak{B}(\mathbb{K})$	context, its concept lattice	§4
G^{red}	transitive reduction of G	what ONTO DAG stores
$\sqcap$	similarity of descriptions	pattern structures, §8.3
$\delta(g)$	the description of object g	pattern structures

Two conventions worth restating

Up is general. Throughout, more general things are drawn higher and arrows point upward, from the special case to the general one. So a cone hangs *down* from a category and a code is read *up* from an item. Some FCA literature draws concept lattices the same way; some data-structure literature draws trees with the root at the top and arrows pointing down. The convention here is fixed and never varies.

The prime is overloaded, deliberately. A' and B' are different maps — one takes objects to attributes, the other attributes to objects — and FCA uses one symbol for both because the argument determines which is meant. This is standard and worth getting used to, since it is what makes expressions like B'' readable.

C The experiments

Every number in this paper is produced by one script, `paper/experiments/fca_experiments.py`. It depends on the Python standard library and on ONTO DAG itself; the FCA half — NextClosure, the derivation operators, stability, the covering relation — is implemented in the file, so no claim

about concept lattices rests on a library the reader would have to take on trust. Where a fast algorithm could be wrong, it is checked against a brute-force oracle in the same run.

Running it

```
$ git clone https://github.com/petfold/ontodag && cd ontodag
$ python3 paper/experiments/fca_experiments.py      # ~1 minute
$ cd paper && make                                   # figures, then the PDF
```

The script writes `experiments/results.txt` (reproduced below), `experiments/explosion.dat` (the data behind Figure 6) and the Graphviz sources for Figures 5, 7, 8 and 9.

What each experiment establishes

#	Establishes	Used in
1	The running example has 10 formal concepts of a possible 32; their supports and stabilities; NextClosure agrees with the oracle.	Tables 1, 2; Fig. 5
2	The same facts as an ONTO DAG: 11 nodes, 17 edges; the query answers; the exact cone sizes.	§5; Fig. 7
3	The Dedekind–MacNeille completion of that ONTO DAG has 17 concepts: 11 nodes, 4 genuine unnamed meets, 2 bookends.	§6; Fig. 8
4	Codes and graphs are mutually recoverable: 200/200 random DAGs reconstruct edge-for-edge from their ancestor-set codes.	Prop. 6.8
5	The blow-up: contranominal scales give 2^n ; random contexts grow superlinearly in density while the stored data does not.	§4.9; Fig. 6
6	A node under A and B is a sibling of the meet, not the meet: $\downarrow AB \subsetneq \downarrow A \cap \downarrow B$.	§6.7; Fig. 9
7	Canonical form: 50 build orders give one graph; 10 give one root hash.	§5.4
8	The implications FCA would find in the same data, and why they are not laws.	§8.1
9	A subsumption certificate verifies against the root alone, in both polarities, and a tampered one is refused.	§10.6

Full output

Reproduced verbatim, so that every figure in the body can be traced to the line that produced it.

```
OntoDAG / FCA experiments
Every number quoted in the paper is produced by this file.

=====
EXPERIMENT 1 -- the running example as a formal context
=====
objects |G| = 6   attributes |M| = 5   density = 0.57

The cross table:
      Flig  Hote  Japa  Book  Refu
outbound.pdf  X    .    X    X    X
inbound.pdf   X    .    X    X    .
kyoto-inn.pdf .    X    X    X    X
rail-pass.pdf .    .    X    X    X
paris-hop.pdf X    .    .    X    .
visa-note.md  .    .    X    .    .
```

NextClosure found 10 formal concepts (brute-force oracle agrees: 10).
Upper bound if every attribute set were closed: $2^5 = 32$.

Every formal concept (extent | intent), with support and stability:

supp=6	stab=0.25	{(no attributes)}
		<- {inbound, kyoto-inn, outbound, paris-hop, rail-pass, visa-note}
supp=5	stab=0.38	{Booked}
		<- {inbound, kyoto-inn, outbound, paris-hop, rail-pass}
supp=5	stab=0.50	{Japan}
		<- {inbound, kyoto-inn, outbound, rail-pass, visa-note}
supp=4	stab=0.38	{Booked, Japan}
		<- {inbound, kyoto-inn, outbound, rail-pass}
supp=3	stab=0.50	{Booked, Flight}
		<- {inbound, outbound, paris-hop}
supp=3	stab=0.62	{Booked, Japan, Refundable}
		<- {kyoto-inn, outbound, rail-pass}
supp=2	stab=0.50	{Booked, Flight, Japan}
		<- {inbound, outbound}
supp=1	stab=0.50	{Booked, Flight, Japan, Refundable}
		<- {outbound}
supp=1	stab=0.50	{Booked, Hotel, Japan, Refundable}
		<- {kyoto-inn}
supp=0	stab=1.00	{Booked, Flight, Hotel, Japan, Refundable}
		<- {(nothing)}

Reduced labelling (each name introduced exactly once):

concept 1:	attributes	['Booked'],	objects	[]
concept 2:	attributes	['Japan'],	objects	['visa-note.md']
concept 4:	attributes	['Refundable'],	objects	['rail-pass.pdf']
concept 5:	attributes	['Hotel'],	objects	['kyoto-inn.pdf']
concept 6:	attributes	['Flight'],	objects	['paris-hop.pdf']
concept 7:	attributes	[],	objects	['inbound.pdf']
concept 8:	attributes	[],	objects	['outbound.pdf']

Concept lattice written to figures/lattice_travel.dot (10 nodes, 13 covering edges).

```
=====
EXPERIMENT 2 -- the same facts as an OntoDAG
=====
nodes = 11   asserted edges = 17
```

A query is one intersection of cones:

get(Japan, Flight	) =	['inbound.pdf', 'outbound.pdf']
get(Japan, Refundable	) =	['kyoto-inn.pdf', 'outbound.pdf', 'rail-pass.pdf']
get(Flight, Hotel	) =	[]
get(Japan	) =	['inbound.pdf', 'kyoto-inn.pdf', 'outbound.pdf', 'rail-pass.pdf', 'visa-note.md']

descendant_count is an exact COUNT(*) per category:

cone(Japan	) =	5
cone(Booked	) =	5
cone(Flight	) =	3
cone(Hotel	) =	1
cone(Refundable	) =	3

OntoDAG written to figures/ontodag_travel.dot

```
=====
EXPERIMENT 3 -- what FCA adds: the Dedekind-MacNeille completion
=====
OntoDAG nodes (excluding the root)      : 11
Concepts of K(G) = (nodes, nodes, reach): 17
```

Every OntoDAG node appears in the completion as a concept whose intent is its own reflexive ancestor set. The EXTRA concepts are the meets the DAG never named:

+ unnamed meet:	everything under {Booked, Flight}
	= {inbound.pdf, outbound.pdf, paris-hop.pdf}
+ unnamed meet:	everything under {Booked, Japan}
	= {inbound.pdf, kyoto-inn.pdf, outbound.pdf, rail-pass.pdf}
+ unnamed meet:	everything under {Booked, Japan, Refundable}
	= {kyoto-inn.pdf, outbound.pdf, rail-pass.pdf}

```

+ unnamed meet: everything under {Booked, Flight, Japan}
    = {inbound.pdf, outbound.pdf}

17 concepts = 11 that are OntoDAG nodes
    + 4 genuine unnamed meets
    + 2 bookends (the top and bottom a
        complete lattice must have, even when nothing
        in the data corresponds to them).

```

Each genuine meet is a combination of categories that happens to be shared by a set of items, but that nobody asserted as a category. FCA materialises them all; OntoDAG materialises none.

Written to figures/dm_completion.dot (asserted edges solid, the meets the completion adds dashed).

```

=====
EXPERIMENT 4 -- codes and DAGs are the same information
=====

```

Claim (SEMANTIC_CODES.md Section 2): the family of reflexive ancestor sets is a lossless, canonical re-encoding of the DAG.
 Test: build random DAGs, take their codes, rebuild the covering relation from the codes alone, and compare with the stored edges.

200/200 random DAGs: covering relation of code containment == the stored transitive reduction, edge for edge. The two directions (DAG -> codes by ancestor closure, codes -> DAG by covering) are mutually inverse, so subsumption really is a subset test.

```

=====
EXPERIMENT 5 -- why OntoDAG refuses to close: the blow-up
=====

```

The contranominal scale $N_n = (\{1..n\}, \{1..n\}, !=)$ is the standard worst case: every attribute subset is closed, so the lattice has 2^n concepts for n objects and n attributes.

n	G = M	concepts	OntoDAG nodes	OntoDAG edges
1	1	2	2	0
2	2	4	4	2
3	3	8	6	6
4	4	16	8	12
5	5	32	10	20
6	6	64	12	30
7	7	128	14	42
8	8	256	16	56
9	9	512	18	72
10	10	1024	20	90
11	11	2048	22	110
12	12	4096	24	132

The OntoDAG columns are linear in the data; the concept column is not. This is the whole reason OntoDAG stores only what was said.

Random contexts (30 objects, 12 attributes, 20 trials each):

density	mean concepts	max concepts	OntoDAG edges (mean)
0.1	21.9	25	35.0
0.2	49.1	66	71.7
0.3	90.0	119	105.2
0.4	183.6	274	145.8
0.5	317.3	460	177.3
0.6	550.3	723	211.3

Written to experiments/explosion.dat for the plot.

```

=====
EXPERIMENT 6 -- OntoDAG is not closed under meets (on purpose)
=====

```

Put AB under {A, B}. Then put X under {A, B} as well.

```

cone(A)      = ['AB', 'X']
cone(B)      = ['AB', 'X']
cone(A) & cone(B) = ['AB', 'X']
cone(AB)     = []

```

So cone(AB) is a STRICT subset of cone(A) & cone(B): X is under both A and B but not under AB. In lattice terms AB is a *sibling* of the meet, never the meet itself. A query planner that substituted AB for the pair {A, B} would silently lose X. In FCA the meet always exists and is the concept with intent closure({A,B}); in OntoDAG a named node is only ever what somebody said it was.

=====

EXPERIMENT 7 -- one canonical form, whatever the history

=====

50 random build orders of the same six facts produced 1 distinct graph(s).

The single canonical edge set:

```
outbound.pdf -> Flight
kyoto-inn.pdf -> Hotel
kyoto-inn.pdf -> Japan
outbound.pdf -> Japan
Flight -> Travel
Hotel -> Travel
```

Note what is NOT there: outbound.pdf -> Travel. It was never asserted, and had it been, transitive reduction would have dropped it, because Flight -> Travel already implies it.

Through the content-addressed store: 10 build orders, 1 distinct root hash(es).
root = 5ff08fead6474d5e9cb1d0466c9a14a57f839f6e5d55813889c22bc44b0c700f

That single 32-byte string is the paper's Section 10 in one line: equal knowledge, equal address, whoever built it and in whatever order.

=====

EXPERIMENT 8 -- implications FCA would find in the same data

=====

An attribute implication B -> C holds when every object with all of B also has all of C. Here are the non-trivial ones with a single-attribute premise, computed by closure:

```
Flight      -> Booked      (support 3)
Hotel       -> Booked, Japan, Refundable (support 1)
Refundable  -> Booked, Japan (support 3)
```

Read as advice to the ontology: 'Flight -> Booked' says every flight in this store happens to be booked. That is a fact about today's population, not a rule -- filing one unbooked flight destroys it. Section 8.1 of the paper is about exactly this distinction, and why such implications belong in a derived, local, regenerable layer rather than in the shared graph.

=====

EXPERIMENT 9 -- an answer a stranger can check

=====

```
root = d53e10b9ee4a06edac177b23d40a3ddab9a8f9a890a4397ac16f50548acc39ea
certificate format : ontodag-is-below-certificate v1
claims            : outbound.pdf is below Travel -> True
records proved    : 4
registry pinned at : 4.1
```

A verifier holding ONLY the root -- no store, no network -- checks:
verify_below(cert, root) = True

Tampering is caught. Flip the claimed answer:
refused: CertificateError

And a negative answer is provable too, in the same way:
outbound.pdf below Hotel? False (verified: False)

This is what Section 10 means by 'trust nobody': the answer travels, the store does not, and a wrong answer cannot verify.

=====

All experiments completed.

D Colophon: how this document is built

The toolchain is recorded because the paper claims reproducibility, and a reproducibility claim that omits the build is incomplete.

Files

ontodag-fca.tex	the document: preamble, styles, and the <code>\input</code> list
sections/*.tex	one file per section, in reading order
ontodag-fca.bib	the bibliography, grouped by literature
experiments/fca_experiments.py	every number in the paper
experiments/results.txt	its output (Appendix C)
experiments/explosion.dat	the data behind Figure 6
figures/*.dot	Graphviz sources; four generated by the script, three hand-written
figures/*.tex	TikZ fragments produced from them by <code>dot2tex</code>
Makefile	the build

The build

```
$ make experiments # rerun the measurements and regenerate the .dot files
$ make figures     # dot2tex: figures/*.dot -> figures/*.tex
$ make             # pdflatex, bibtex, pdflatex, pdflatex
```

Requirements: `pdflatex` and `bibtex` (TeX Live 2023 or later, with `tikz`, `pgfplots`, `tcolorbox`, `natbib`, `stmaryrd`), `graphviz` for `dot`, `dot2tex`, and Python 3 with `ONTODAG importable` (the script inserts `../src` on the path, so a checkout suffices; no installation needed). The certificate experiment additionally needs `recordstore`, and skips with a printed note if it is absent.

How the figures are made

Two kinds, deliberately.

Graph figures come from Graphviz. A `.dot` file is laid out by `dot` and converted to a TikZ fragment by `dot2tex` (`--format tikz --codeonly --texmode raw`), which emits bare `\node` and `\draw` commands with coordinates. The paper wraps each fragment in its own `tikzpicture` via the `\dotfig` macro, so scale, fonts and node styles are the document’s decision rather than Graphviz’s. Two consequences worth recording for anyone reusing the setup:

- In `raw` `texmode` a label is LaTeX, so `&` must be avoided or escaped and a line break must reach LaTeX as `\\` — which means four backslashes in the `.dot` source. The wrapper sets `align=center` on every node so that `\\` is legal.
- Graphviz colour attributes pass through as raw names, and HSB triples (Graphviz’s default numeric colour syntax) are not a colour model PGF understands. The figures therefore use no colour at all and distinguish node roles by shape and border count — which also means they survive greyscale printing.

Conceptual figures are hand-written TikZ, because their content is an argument rather than data: Figures 1, 2, 4, 10 and 12. The chart in Figure 6 is `pgfplots` reading the generated `.dat` file, so it cannot disagree with the text.

Relationship to the companion paper

A companion manuscript in the same directory, `main.tex` (Földiák, 2026c), presents ONTODAG as a semantic layer for distributed storage: the data structure, the persistence mapping onto Swarm, the multi-writer merge, and the MDL learning direction. It overlaps this one in Sections 5 and 11 and is the better starting point for a reader who wants the systems story rather than the order theory. This paper is the theoretical companion: what the structure *is*, in the terms of the field that has studied such structures for forty years.

Provenance of this document

Drafted in August 2026 with Claude (Anthropic) as a writing and analysis collaborator, working from the ONTODAG repository’s design records — in particular the notes on semantic codes, the higher-layer contract, the database-direction walls, and the historical survey of philosophical languages, all of which are cited in the body where their arguments are used. The experiments were written for this paper and run before the claims that quote them; the transcripts in Sections 5 and 10 are real sessions, not illustrations.